Pruning the Search Tree

Early termination, bounds, symmetry, and duplicate states

CSCI 235 – Data Structures

Hunter College, Fall 2026

Class 16 Thursday, October 29, 2026

Today

1. The search is a **tree**. Two ways to visit fewer nodes: **early termination** and **pruning**.
2. Pruning with a **bound**: can the rest still reach the target?
3. When pruning is **safe**, and when it quietly loses answers.
4. Make 24 by **pair selection**, **symmetry**, and **duplicate states**.
5. Duplicate letters: skipping repeated branches in the arrangement generator.

- **Project 3 (Make 24 Solver)** is posted today. Today's ideas are tools for it; the handout has the exact requirements and the grading rules.
- **Exercise 7 (Permutation Trace)** is due **Tuesday, Nov 3**.

Recap: Subset Sum, Write It Yourself

Does some subset of `v[k..]` add up to `rem`? At each number we **take** it or **skip** it. Example: `v = {4, 3, 2}`, target 5. Fill each blank; hints appear one at a time.

`bool find(v, k, rem, chosen):`

	Pseudo-code	Hint
1	`if (_____) return true`	
2	`if (_____ or _____) return false`	
3	`_____ v[k] to chosen`	
4	`if (find(_____, _____)) return true`	
5	`_____ the last item of chosen`	
6	`return find(_____, _____)`	

Recap: Subset Sum, Write It Yourself

Does some subset of `v[k..]` add up to `rem`? At each number we **take** it or **skip** it. Example: `v = {4, 3, 2}`, target 5. Fill each blank; hints appear one at a time.

`bool find(v, k, rem, chosen):`

	Pseudo-code	Hint
1	`if (_____) return true`	When is a subset found? What must `rem` be?
2	`if (_____ or _____) return false`	
3	`_____ v[k] to chosen`	
4	`if (find(_____, _____)) return true`	
5	`_____ the last item of chosen`	
6	`return find(_____, _____)`	

Recap: Subset Sum, Write It Yourself

Does some subset of `v[k..]` add up to `rem`? At each number we **take** it or **skip** it. Example: `v = {4, 3, 2}`, target 5. Fill each blank; hints appear one at a time.

`bool find(v, k, rem, chosen):`

	Pseudo-code	Hint
1	`if (_____) return true`	When is a subset found? What must `rem` be?
2	`if (_____ or _____) return false`	Two dead ends: look at `k` and at `rem`.
3	`_____ v[k] to chosen`	
4	`if (find(_____, _____)) return true`	
5	`_____ the last item of chosen`	
6	`return find(_____, _____)`	

Recap: Subset Sum, Write It Yourself

Does some subset of `v[k..]` add up to `rem`? At each number we **take** it or **skip** it. Example: `v = {4, 3, 2}`, target 5. Fill each blank; hints appear one at a time.

`bool find(v, k, rem, chosen):`

	Pseudo-code	Hint
1	`if (_____) return true`	When is a subset found? What must `rem` be?
2	`if (_____ or _____) return false`	Two dead ends: look at `k` and at `rem`.
3	`_____ v[k] to chosen`	Which `vector` call adds at the back?
4	`if (find(_____, _____)) return true`	
5	`_____ the last item of chosen`	
6	`return find(_____, _____)`	

Recap: Subset Sum, Write It Yourself

Does some subset of `v[k..]` add up to `rem`? At each number we **take** it or **skip** it. Example: `v = {4, 3, 2}`, target 5. Fill each blank; hints appear one at a time.

`bool find(v, k, rem, chosen):`

	Pseudo-code	Hint
1	`if (_____) return true`	When is a subset found? What must `rem` be?
2	`if (_____ or _____) return false`	Two dead ends: look at `k` and at `rem`.
3	`_____ v[k] to chosen`	Which `vector` call adds at the back?
4	`if (find(_____, _____)) return true`	Take `v[k]`: next `k`? new `rem`?
5	`_____ the last item of chosen`	
6	`return find(_____, _____)`	

Recap: Subset Sum, Write It Yourself

Does some subset of `v[k..]` add up to `rem`? At each number we **take** it or **skip** it. Example: `v = {4, 3, 2}`, target 5. Fill each blank; hints appear one at a time.

`bool find(v, k, rem, chosen):`

	Pseudo-code	**Hint**
1	`if (_____) return true`	When is a subset found? What must `rem` be?
2	`if (_____ or _____) return false`	Two dead ends: look at `k` and at `rem`.
3	`_____ v[k] to chosen`	Which `vector` call adds at the back?
4	`if (find(_____, _____)) return true`	Take `v[k]`: next `k`? new `rem`?
5	`_____ the last item of chosen`	The take failed. Which call undoes step 3?
6	`return find(_____, _____)`	

Recap: Subset Sum, Write It Yourself

Does some subset of `v[k..]` add up to `rem`? At each number we **take** it or **skip** it. Example: `v = {4, 3, 2}`, target 5. Fill each blank; hints appear one at a time.

`bool find(v, k, rem, chosen):`

	Pseudo-code	**Hint**
1	`if (_____) return true`	When is a subset found? What must `rem` be?
2	`if (_____ or _____) return false`	Two dead ends: look at `k` and at `rem`.
3	`_____ v[k] to chosen`	Which `vector` call adds at the back?
4	`if (find(_____, _____)) return true`	Take `v[k]`: next `k`? new `rem`?
5	`_____ the last item of chosen`	The take failed. Which call undoes step 3?
6	`return find(_____, _____)`	Skip `v[k]`: which `k` and `rem`?

Recap: Subset Sum, Write It Yourself

Does some subset of `v[k..]` add up to `rem`? At each number we **take** it or **skip** it. Example: `v = {4, 3, 2}`, target 5. Fill each blank; hints appear one at a time.

`bool find(v, k, rem, chosen):`

	Pseudo-code	Hint
1	`if (_____) return true`	When is a subset found? What must `rem` be?
2	`if (_____ or _____) return false`	Two dead ends: look at `k` and at `rem`.
3	`_____ v[k] to chosen`	Which `vector` call adds at the back?
4	`if (find(_____, _____)) return true`	Take `v[k]`: next `k`? new `rem`?
5	`_____ the last item of chosen`	The take failed. Which call undoes step 3?
6	`return find(_____, _____)`	Skip `v[k]`: which `k` and `rem`?

Check yourself: trace `v = {4, 3, 2}`, target 5: **9 calls**, ending with `chosen = {3, 2}`.

Question: Is the Whole Tree Needed?

Last time we searched for a subset of `v = {4, 3, 2}` that adds up to **5**. At every number the search either **takes** it or **skips** it, so the full tree has $1 + 2 + 4 + 8 = \mathbf{15}$ nodes.

(a) Suppose we take 4 and then take 3. The remaining target is $\texttt{rem} = -2$. Should we explore the children of this node?
(b) Suppose we have just found $\{3, 2\}$. The question was "is there a subset?" Should we keep searching?

Question: Is the Whole Tree Needed?

Last time we searched for a subset of `v = {4, 3, 2}` that adds up to **5**. At every number the search either **takes** it or **skips** it, so the full tree has $1 + 2 + 4 + 8 = \mathbf{15}$ nodes.

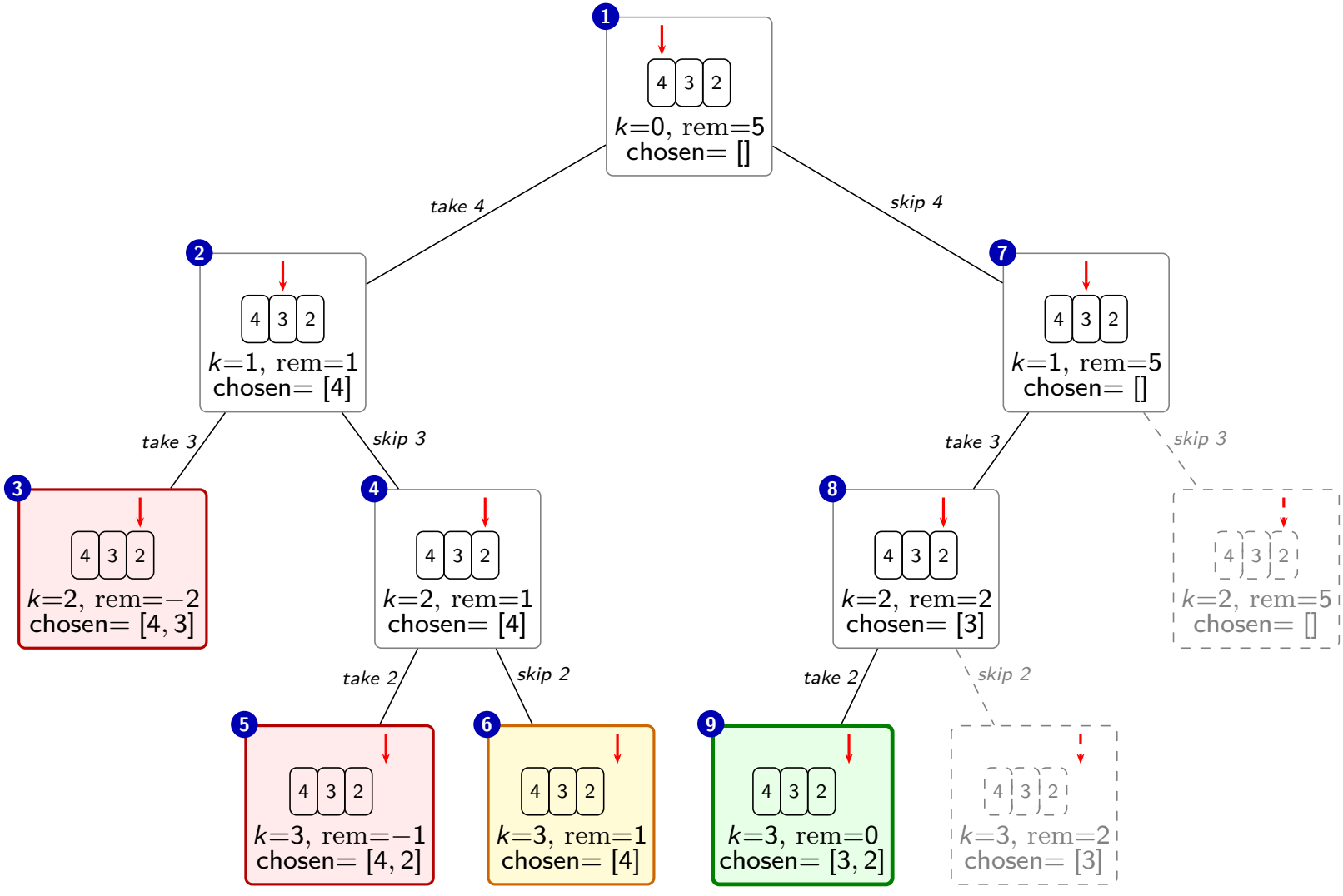
(a) Suppose we take 4 and then take 3. The remaining target is `rem` $= -2$. Should we explore the children of this node?
(b) Suppose we have just found $\{3, 2\}$. The question was "is there a subset?" Should we keep searching?

(a) No. All the numbers are non-negative, so taking more only makes `rem` smaller. A negative `rem` can never return to 0. Skip the whole subtree: **pruning**.

(b) No. One example answers "yes". Return `true` right away, all the way up the stack: **early termination**.

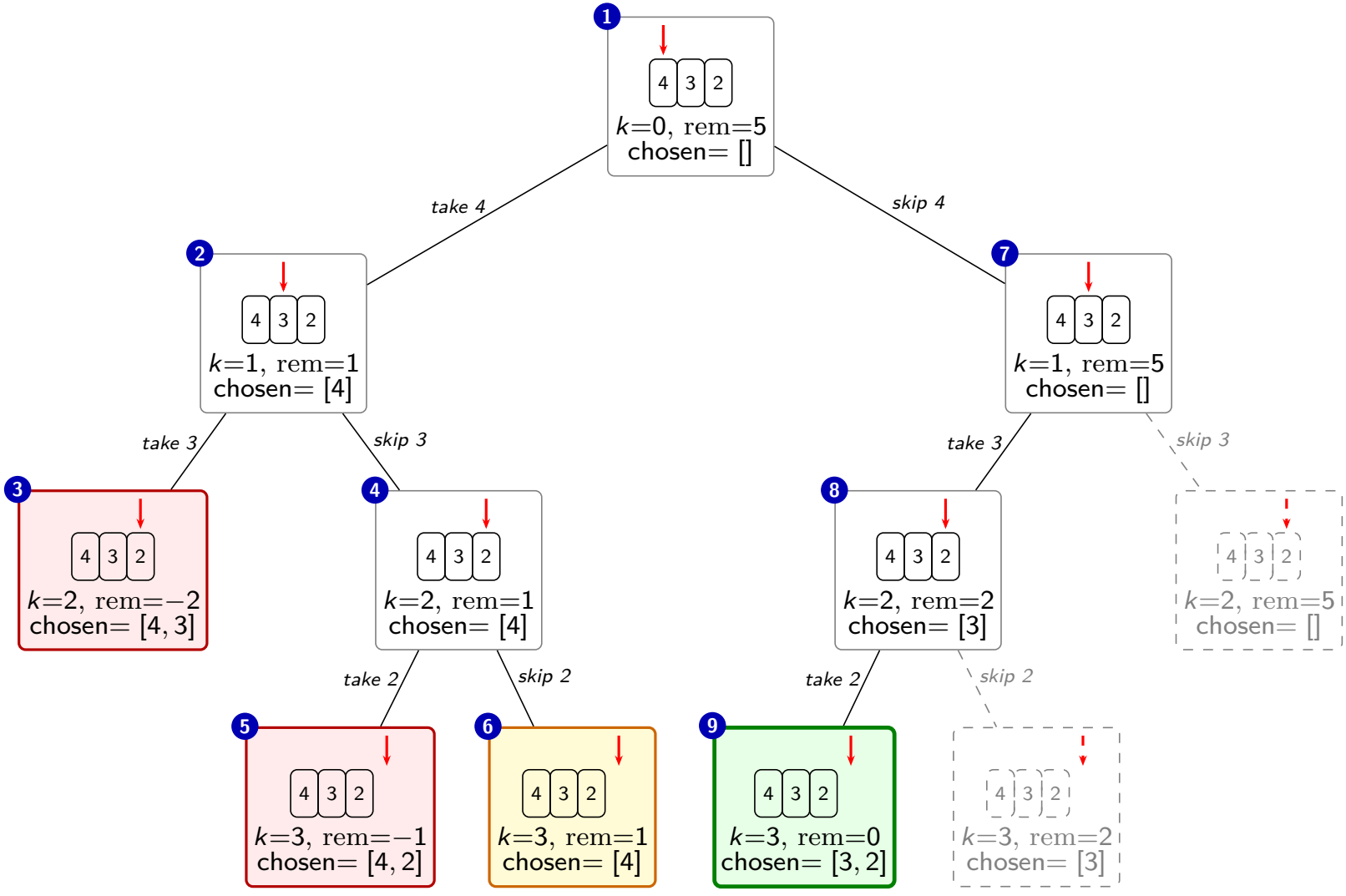
The Call Tree, With Both Savings

`v = {4, 3, 2}`, target 5. Each call is a node: the array with an arrow at `v[k]`, then `k`, `rem`, and `chosen`. Blue numbers give the **order of the calls**. Gray dashed nodes are never visited.



The Call Tree, With Both Savings

9 calls instead of 15. The red node at call 3 (`rem` < 0) is cut by **pruning**: its two children are never created. The green node ends the search: the dashed gray nodes and everything below them (4 nodes) are saved by **early termination**. The yellow node ran out of numbers with `rem` $\neq 0$.



Two Tools, Two Different Questions

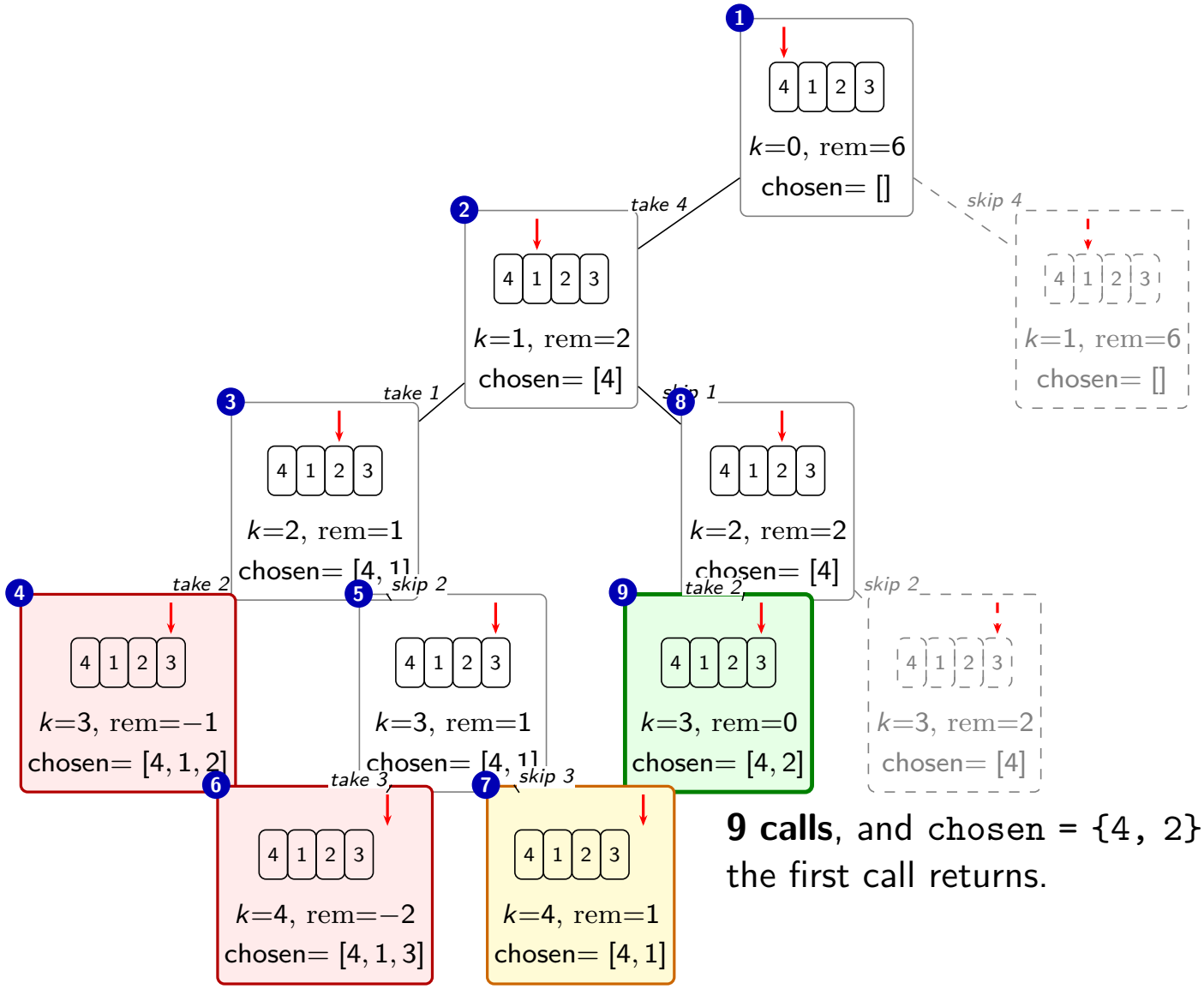
	Early termination	**Pruning**
Asks	"Have I *already found* what I need?"	"Can this branch *ever* succeed?"
Action	Return `true` up the stack.	Return `false` for this branch.
Code	`if (explore(...)) return true;`	`if (rem < 0) return false;`
Valid when	The question is "does one exist?" (not "list all").	You can **prove** no goal is below.

Analogy: finding the exit of a building. Early termination: you stop as soon as you see the exit. Pruning: a door has a sign "flooded, nothing beyond" and you do not open it.

Careful: "count all solutions" and "print all solutions" must **not** stop early. Pruning is still allowed there.

`v` Does Not Need to Be Sorted

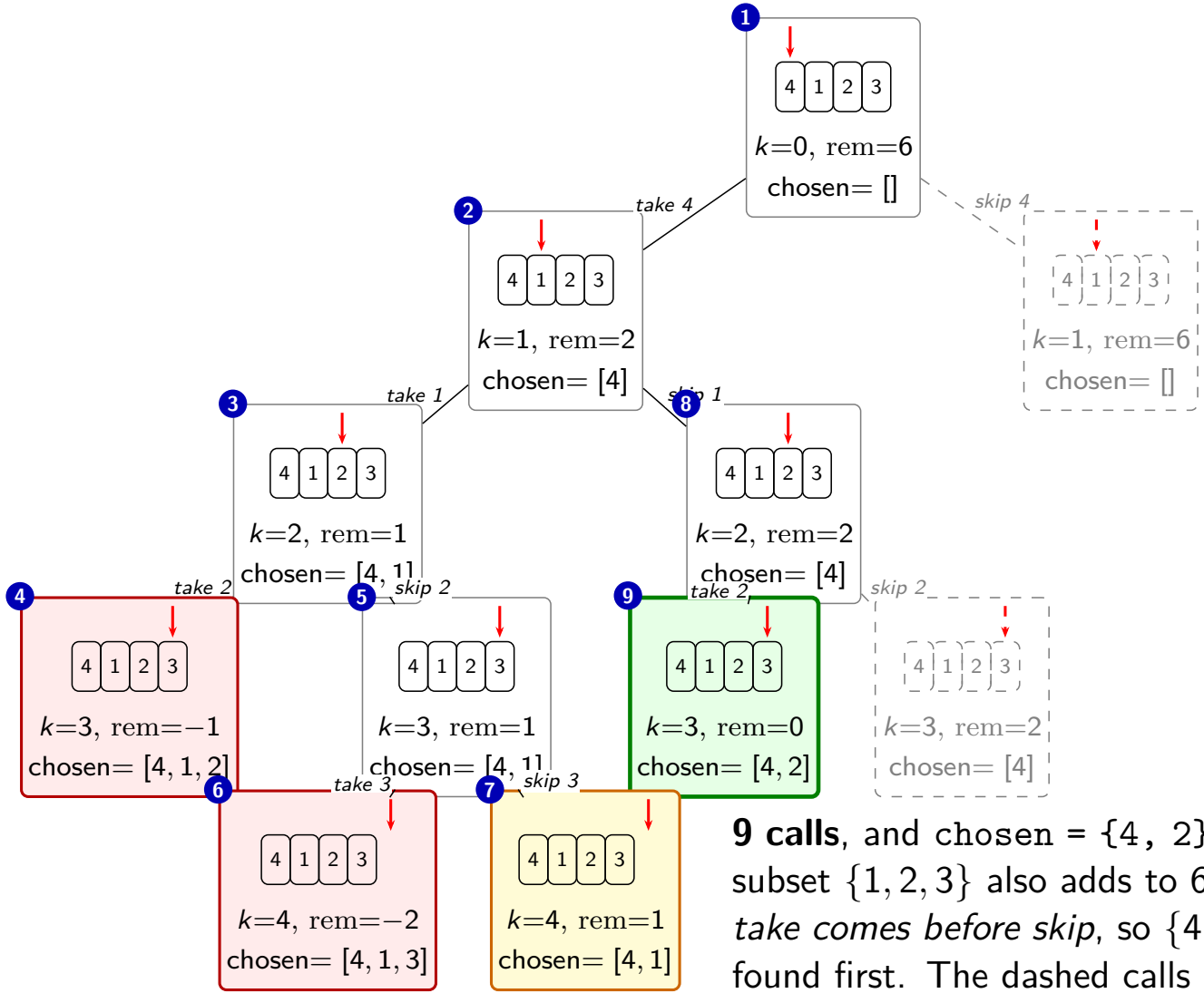
`v = {4, 1, 2, 3}`, target 6. Left edge = take, right edge = skip.



9 calls, and `chosen = {4, 2}` when the first call returns.

`v` Does Not Need to Be Sorted

`v = {4, 1, 2, 3}`, target 6. Left edge = take, right edge = skip.



9 calls, and `chosen = {4, 2}`. The subset $\{1, 2, 3\}$ also adds to 6, but *take comes before skip*, so $\{4, 2\}$ is found first. The dashed calls are never created.

Question: A Bound on What Is Left

Now `v = {9, 8, 7, 6, 5, 4, 3, 2, 1}`, and the target is **46**. All nine numbers add up to **45**.

(a) How many calls does the full take/skip search make? Can any subset reach 46?

(b) What cheap test would stop the search at the very first call?

Question: A Bound on What Is Left

Now `v = {9, 8, 7, 6, 5, 4, 3, 2, 1}`, and the target is **46**. All nine numbers add up to **45**.

(a) How many calls does the full take/skip search make? Can any subset reach 46?

(b) What cheap test would stop the search at the very first call?

(a) $2^{10} - 1 = 1023$ calls, and no subset can reach 46 (the whole set only reaches 45).

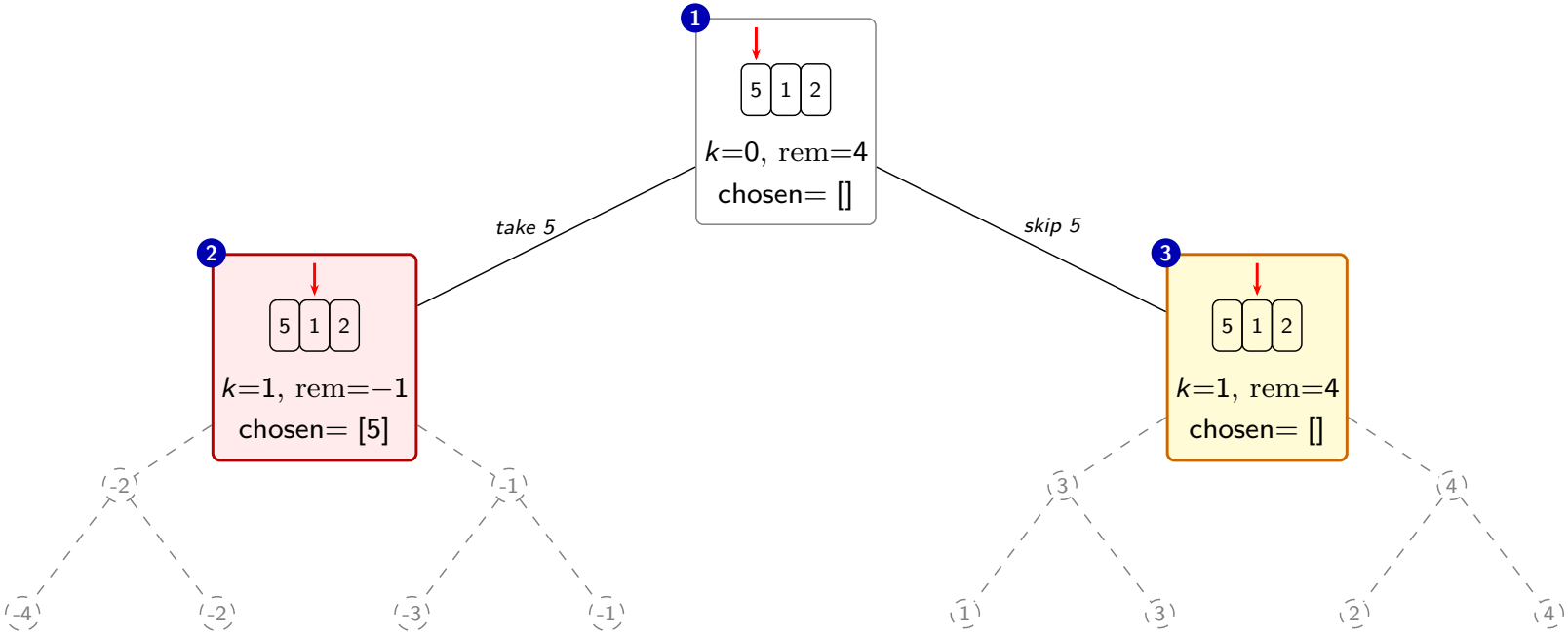
(b) Keep `suffix[k]`, the sum of `v[k..n-1]` (and `suffix[n] = 0`: nothing left). If `rem > suffix[k]`, even taking **everything** left falls short, so prune:

```
if (rem == 0) { return true; }
if (k == n || rem < 0 || rem > suffix[k]) { return false; } // bound
```

Result: **1 call** instead of 1023. For target 20 (a real solution) the calls are $1023 \to 244 \to 12$.

A Small Bound Example

`v = {5, 1, 2}`, target 4. Then `suffix = {8, 3, 2, 0}`. No subset adds to 4, so the answer is `false`.



Call 2 stops because `rem < 0`. Call 3 stops because `rem = 4 > suffix[1] = 3`: even taking both 1 and 2 reaches only 3. The dashed circles (each shows `rem`) are the calls that never happen.

Without any pruning: $1 + 2 + 4 + 8 = 2^4 - 1 = \mathbf{15}$ calls. With both tests: **3** calls.

Measured: How Much Do the Savings Add Up To?

Same `v = {9, 8, 7, 6, 5, 4, 3, 2, 1}`. Counting the calls of `find`:

Version	**target 20**	**target 46**	**target 100**
Full tree (no savings)	1023	1023	1023
+ early termination			
+ prune `rem < 0`			
+ prune `rem > suffix[k]`			

Measured: How Much Do the Savings Add Up To?

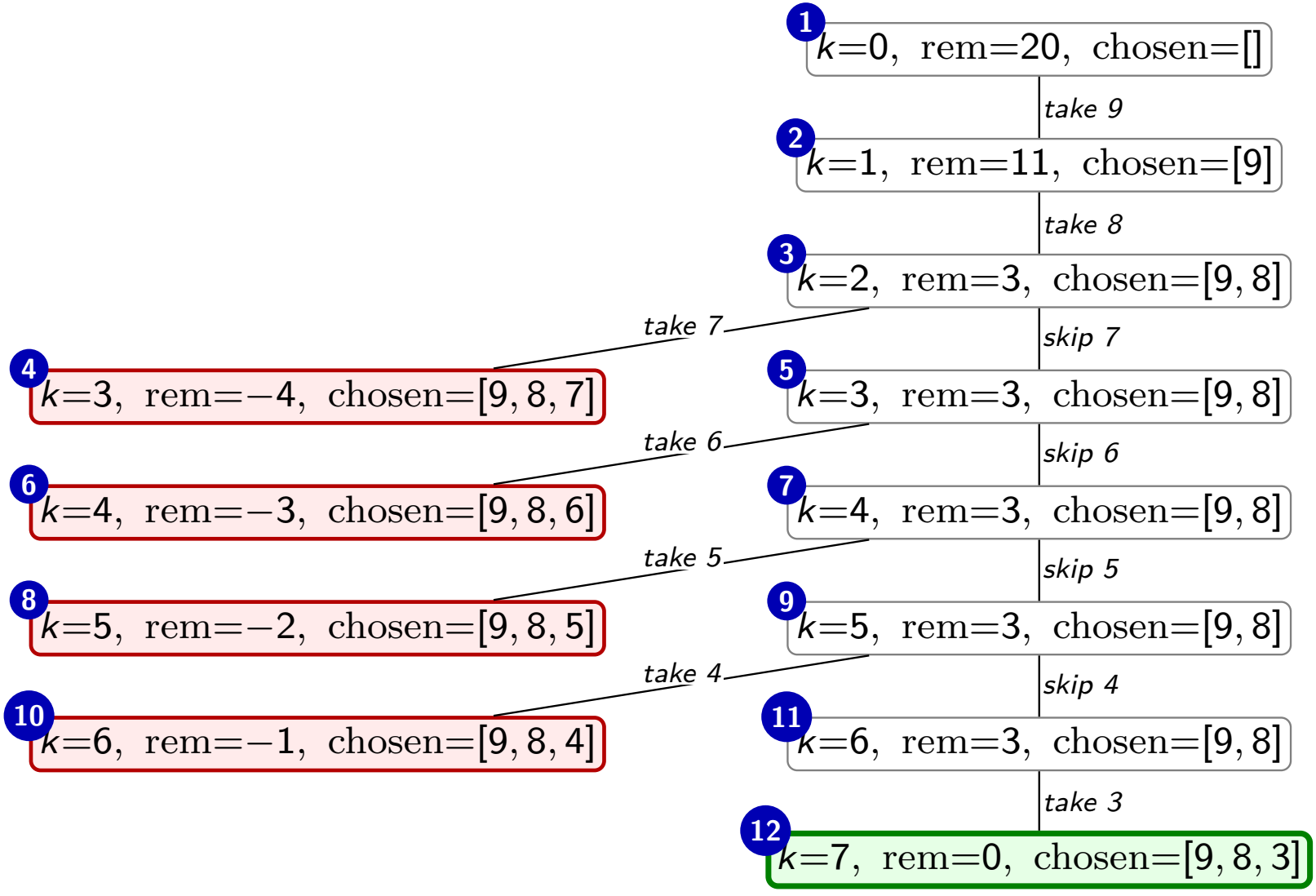
Same `v = {9, 8, 7, 6, 5, 4, 3, 2, 1}`. Counting the calls of `find`:

Version	**target 20**	**target 46**	**target 100**
Full tree (no savings)	1023	1023	1023
+ early termination	244	1023	1023
+ prune `rem < 0`	12	1023	1023
+ prune `rem > suffix[k]`	12	1	1

Early termination only helps when a solution **exists**. The `rem < 0` prune helps when we overshoot. A target that is **too large** is caught only by the bound. Each test removes a different kind of waste.

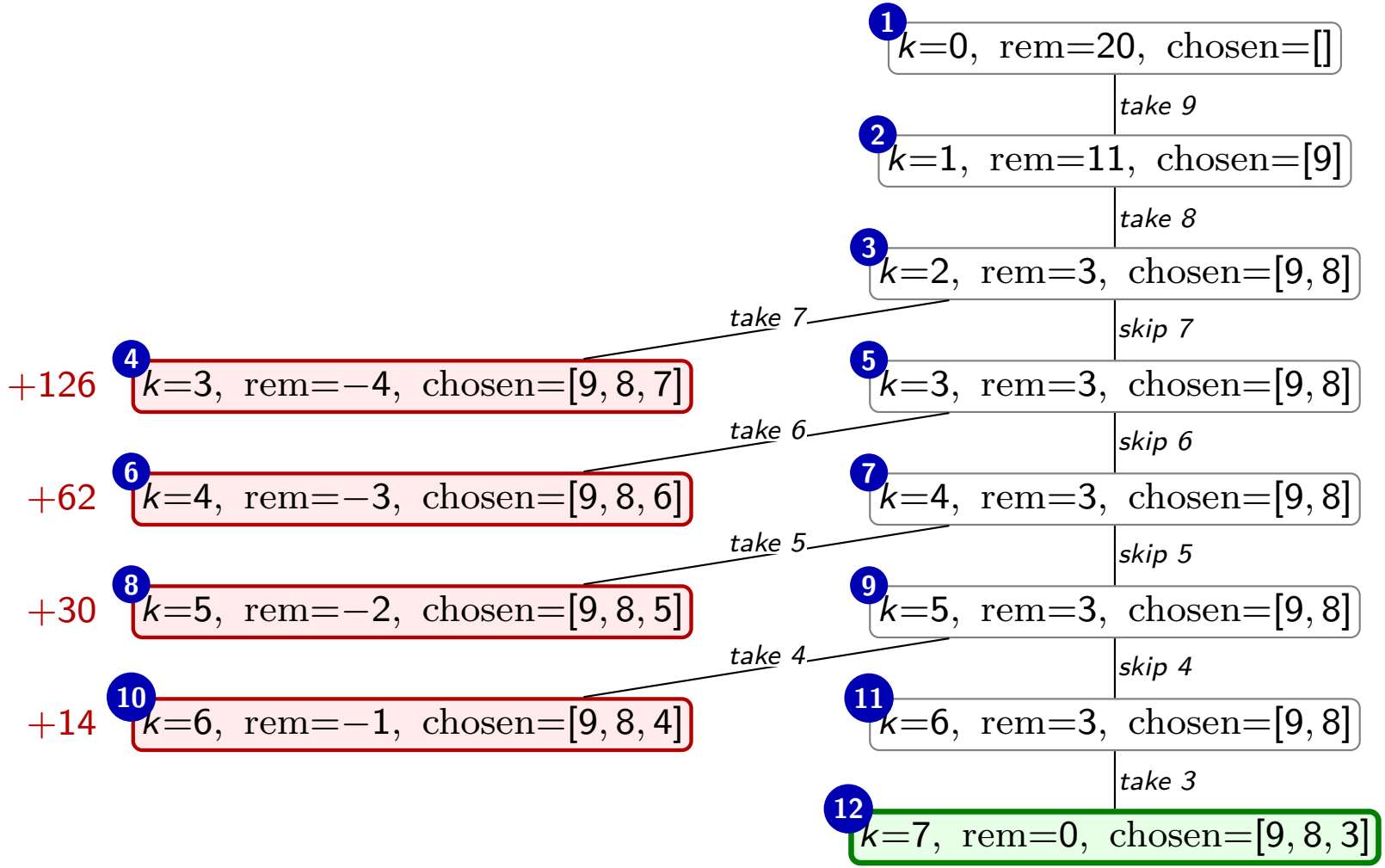
Target 20: the 12 calls when `rem < 0` is tested

`v = {9, 8, 7, 6, 5, 4, 3, 2, 1}`. Red calls overshoot, so they return at once. Green is the goal.



Target 20: the 12 calls when `rem < 0` is tested

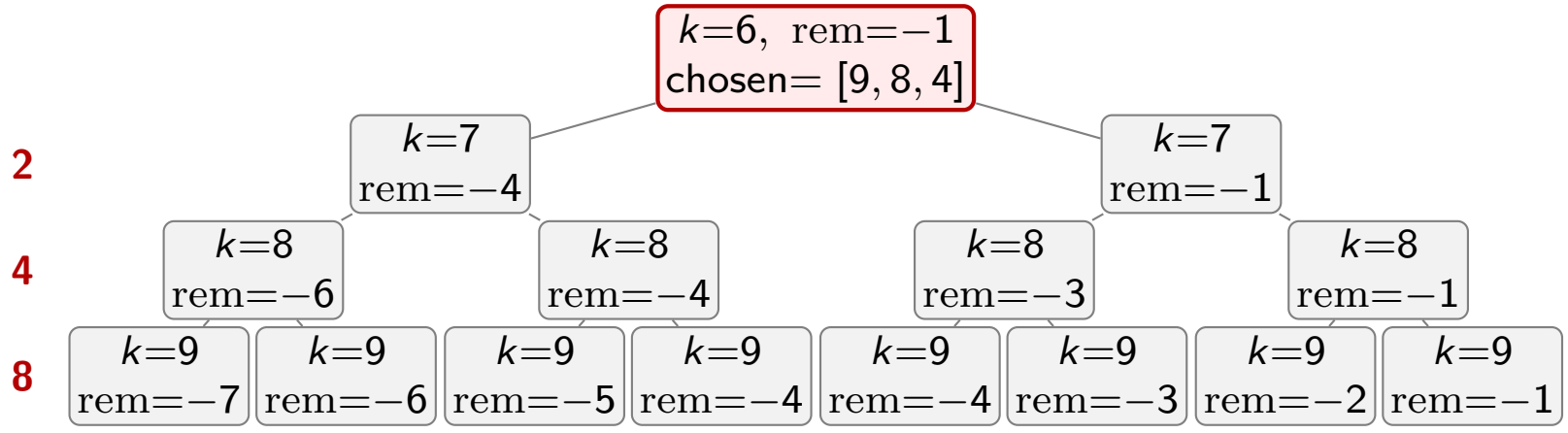
`v = {9, 8, 7, 6, 5, 4, 3, 2, 1}`. Red calls overshoot, so they return at once. Green is the goal.



Without the `rem < 0` test, each red call keeps going through all the numbers that are left. The numbers beside the red calls are the **extra** calls each one would cause.

Without `rem < 0`: the branch "choose 4" keeps going

Left child = take, right child = skip. The number at each level is how many calls that level has.

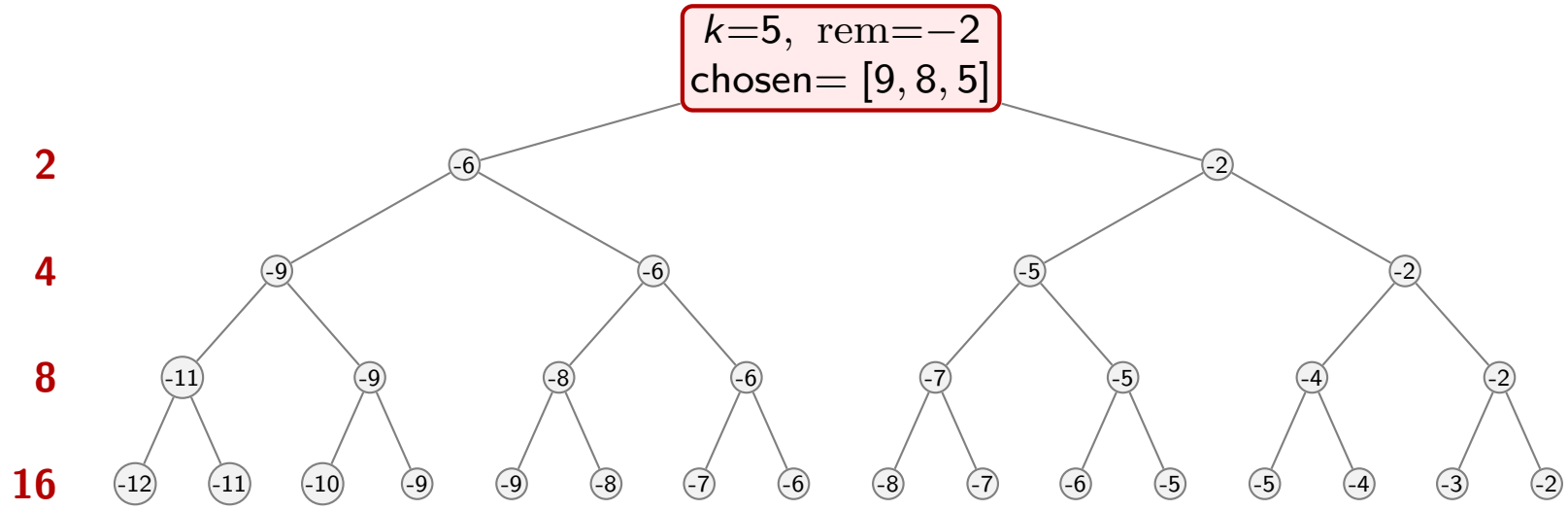


Three numbers are left (3, 2, 1), so the tree below the red call has three levels.

$$2 + 4 + 8 = 2^4 - 2 = 14 \textbf{ extra calls}$$

Without `rem < 0`: the branch "choose 5" keeps going

Left child = take, right child = skip. The number at each level is how many calls that level has.

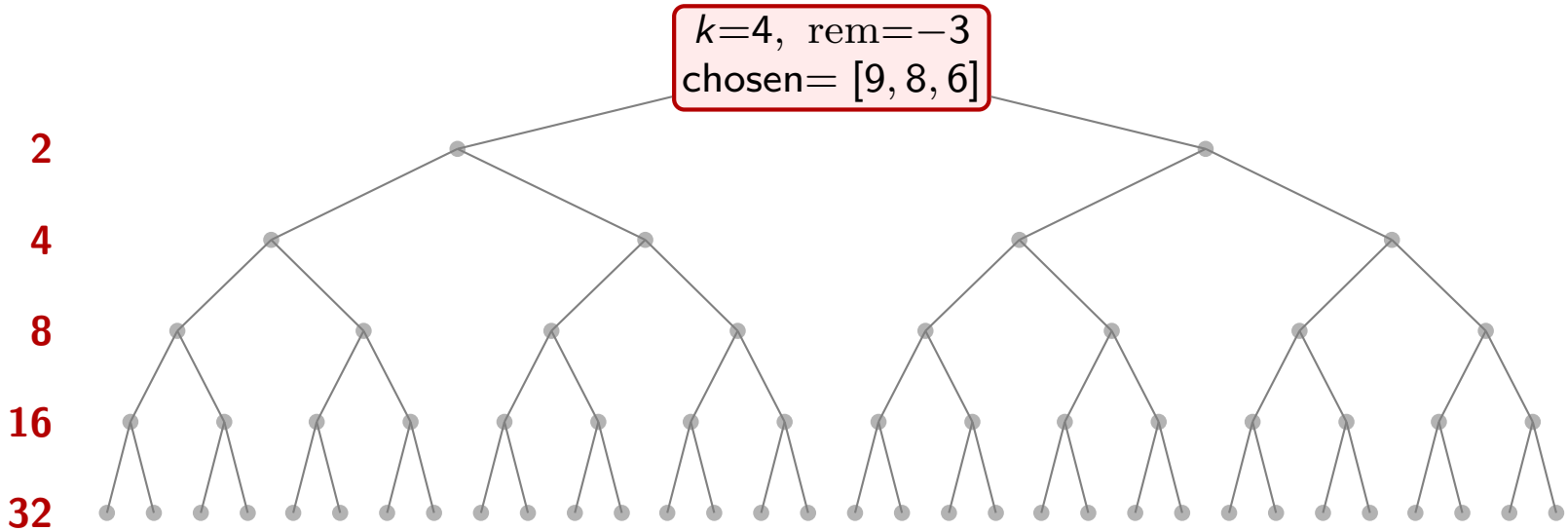


Four numbers are left, so four levels: 30 calls below the red call. The number in each circle is `rem`.

$$2 + 4 + 8 + 16 = 2^5 - 2 = 30 \textbf{ extra calls}$$

Without `rem < 0`: the branch "choose 6" keeps going

Left child = take, right child = skip. The number at each level is how many calls that level has.

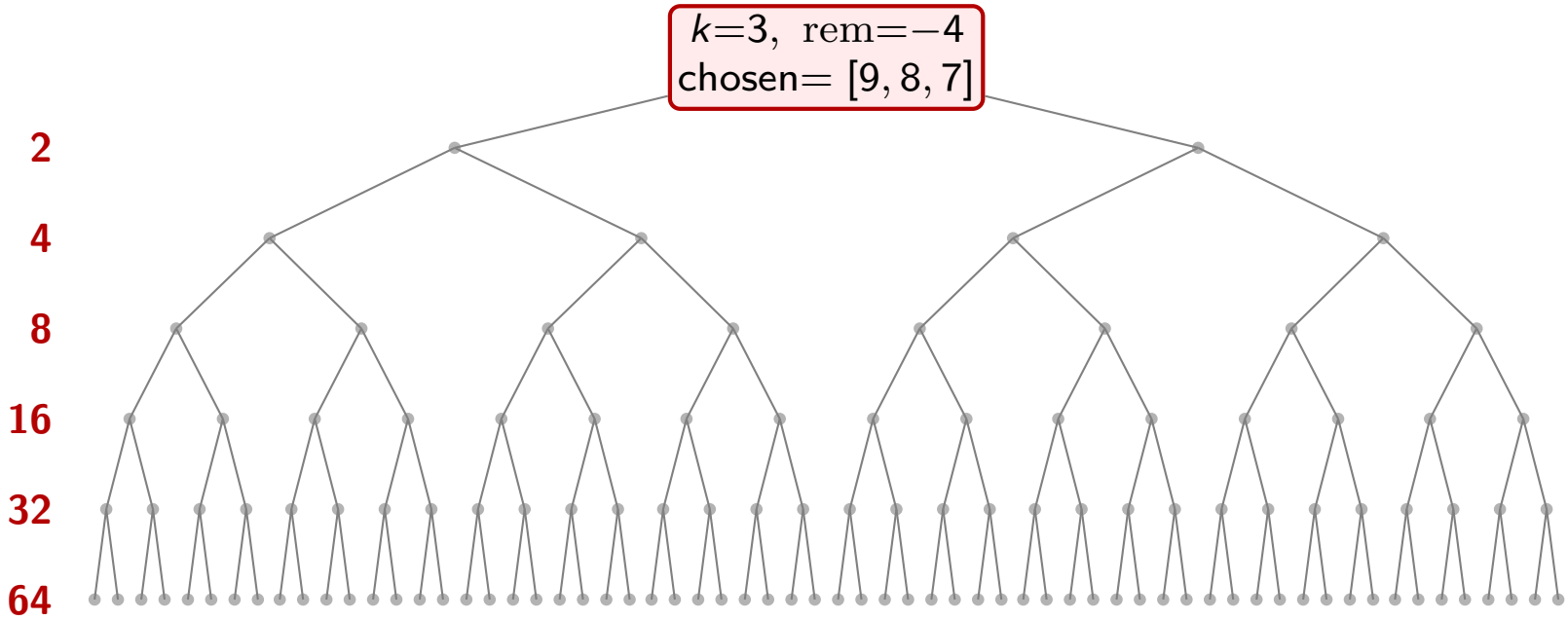


Five numbers are left, so five levels. Each dot is one call, and every one of them is wasted.

$$2 + 4 + 8 + 16 + 32 = 2^6 - 2 = 62 \textbf{ extra calls}$$

Without `rem < 0`: the branch "choose 7" keeps going

Left child = take, right child = skip. The number at each level is how many calls that level has.



Six numbers are left, so six levels, and 64 calls on the bottom row alone.

$$2 + 4 + 8 + 16 + 32 + 64 = 2^7 - 2 = 126 \textbf{ extra calls}$$

Where the 244 Calls Come From

Target 20, early termination only (no `rem < 0` test):

Part of the tree	**Formula**	**Calls**
Same calls as the `rem < 0` version		12
Below the red call "choose 4"	$2^4 - 2$	14
Below the red call "choose 5"	$2^5 - 2$	30
Below the red call "choose 6"	$2^6 - 2$	62
Below the red call "choose 7"	$2^7 - 2$	126
Total	$12 + 14 + 30 + 62 + 126$	**244**

A single `rem < 0` test cuts all four red subtrees at their roots: $244 \to 12$.

Pruning Is a Promise

A pruning test says: "**no goal can be below this node**." If that is false, the search **silently misses answers** and still looks correct.

Question. The pruned `find` (with `rem < 0` returning `false`) is run on `v = {5, -3}` with target **2**. The subset $\{5, -3\}$ adds up to 2. What does it return?

Pruning Is a Promise

A pruning test says: "**no goal can be below this node**." If that is false, the search **silently misses answers** and still looks correct.

Question. The pruned `find` (with `rem < 0` returning `false`) is run on `v = {5, -3}` with target **2**. The subset $\{5, -3\}$ adds up to 2. What does it return?

It returns `false`. Taking 5 gives `rem` $= 2 - 5 = -3 < 0$, so the branch is cut. But the next number, -3, would have brought `rem` back to 0. The prune assumed the numbers are **non-negative**.

Rule: before adding a pruning test, say out loud *why* it can never cut a goal. If the reason depends on an assumption (non-negative numbers, sorted data), the assumption must be true for **every** input.

Make 24: Start With an Example

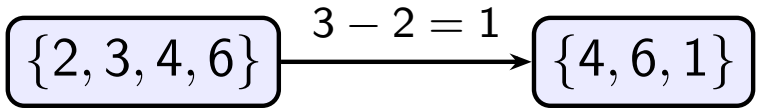
Four numbers are on the table: $\{2,\ 3,\ 4,\ 6\}$. A **move** picks two of them and replaces them by **one** result $(+,\ -,\ \times,\ \div)$. Can you reach 24? **Count the numbers on the table after each move.**

$$\{2, 3, 4, 6\}$$

4 numbers

Make 24: Start With an Example

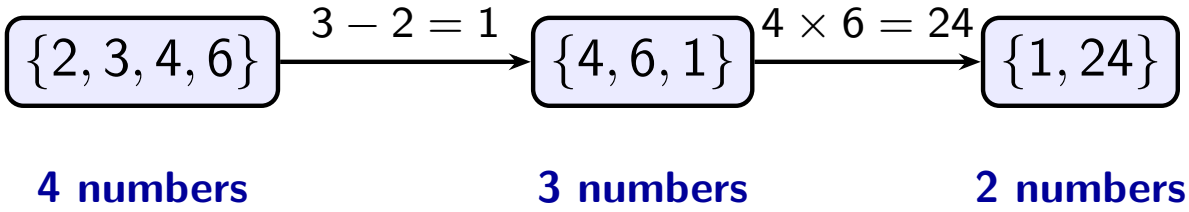
Four numbers are on the table: $\{2,\ 3,\ 4,\ 6\}$. A **move** picks two of them and replaces them by **one** result $(+,\ -,\ \times,\ \div)$. Can you reach 24? **Count the numbers on the table after each move.**



4 numbers **3 numbers**

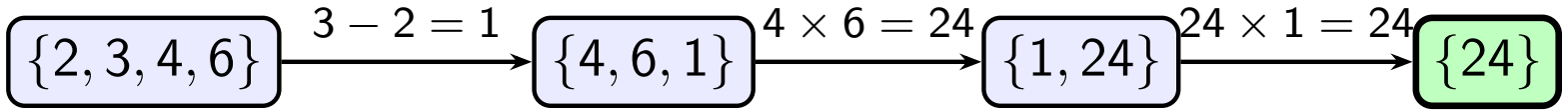
Make 24: Start With an Example

Four numbers are on the table: $\{2,\ 3,\ 4,\ 6\}$. A **move** picks two of them and replaces them by **one** result $(+,\ -,\ \times,\ \div)$. Can you reach 24? **Count the numbers on the table after each move.**



Make 24: Start With an Example

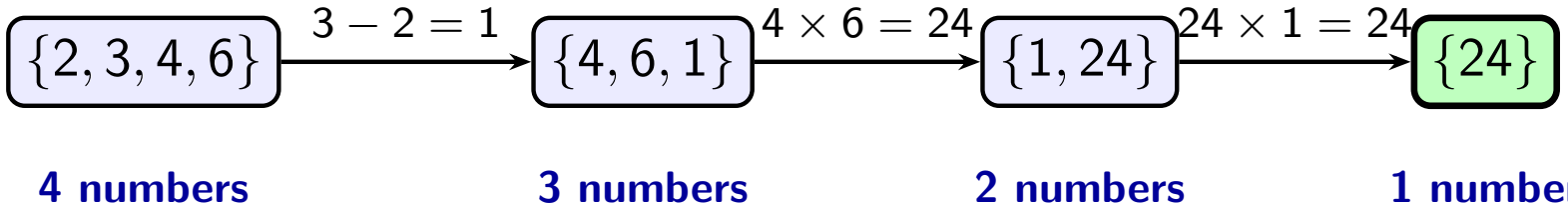
Four numbers are on the table: $\{2,\ 3,\ 4,\ 6\}$. A **move** picks two of them and replaces them by **one** result $(+,\ -,\ \times,\ \div)$. Can you reach 24? **Count the numbers on the table after each move.**



4 numbers **3 numbers** **2 numbers** **1 number**

Make 24: Start With an Example

Four numbers are on the table: $\{2,\ 3,\ 4,\ 6\}$. A **move** picks two of them and replaces them by **one** result $(+,\ -,\ \times,\ \div)$. Can you reach 24? **Count the numbers on the table after each move.**



The idea: every move turns a problem with n numbers into the *same* problem with $n-1$ numbers. So `solve` can call itself on a **smaller vector**, and it stops when only **one** number is left. Next, we build the function one step at a time. At each step, choose your answer first.

Step 1: When Only One Number Is Left

Question. `solve(nums)` is called with `nums = {x}`, one number. There is no pair left to merge. What should it do?

(A) `return true;`

(B) `return nums[0] == 24;`

(C) `return std::abs(nums[0] - 24) < 1e-9;`

(D) Pick another pair and keep merging.

```
bool solve(const std::vector<double>& nums) {
    // ??? the smallest problem
}
```

Step 1: When Only One Number Is Left

Question. `solve(nums)` is called with `nums = {x}`, one number. There is no pair left to merge. What should it do?

(A) `return true;`

(B) `return nums[0] == 24;`

(C) `return std::abs(nums[0] - 24) < 1e-9;`

(D) Pick another pair and keep merging.

(C). The only question left is "is it 24?" Compare `doubles` with a **tolerance**: floating-point rounding can leave a result that is mathematically 24 slightly above or below 24, so `== 24` (B) may miss a real solution. (A) would accept any single number. (D) is impossible: there is no pair.

```
bool solve(const std::vector<double>& nums) {
    if (nums.size() == 1) { return std::abs(nums[0] - 24) < 1e-9; } // goal
    // ... the rest of the function
}
```

Step 2: Choose Every Pair

Question. With $n = 4$ numbers there are $\binom{4}{2} = 6$ **unordered** pairs $\{a, b\}$, and each pair should be visited once. Which loops do that? (`i` and `j` are positions in `nums`.)

(A) `i` from 0 to $n - 1$, `j` from `i + 1` to $n - 1$

(B) `i` from 0 to $n - 1$, `j` from 0 to $n - 1$

(C) `i` from 0 to $n - 1$, `j` from `i` to $n - 1$

(D) `i` from 0 to $n - 2$, and only `j = i + 1`

```
bool solve(const std::vector<double>& nums) {
    if (nums.size() == 1) { return std::abs(nums[0] - 24) < 1e-9; } // goal
    for (std::size_t i = ...; i < ...; ...) {
        for (std::size_t j = ...; j < ...; ...) {     // unordered pair
            // ???
        }
    }
}
```

Step 2: Choose Every Pair

Question. With $n = 4$ numbers there are $\binom{4}{2} = 6$ **unordered** pairs $\{a, b\}$, and each pair should be visited once. Which loops do that? (`i` and `j` are positions in `nums`.)

(A) **`i` from 0 to $n-1$, `j` from `i + 1` to $n-1$**

(B) `i` from 0 to $n-1$, `j` from 0 to $n-1$

(C) `i` from 0 to $n-1$, `j` from `i` to $n-1$

(D) `i` from 0 to $n-2$, and only `j = i + 1`

(A). $3 + 2 + 1 = 6$ pairs. (B) visits $4 \times 4 = 16$ pairs: it pairs a number with itself and visits each pair twice. (C) still pairs a number with itself ($i = j$). (D) only visits neighbors: 3 pairs.

```
bool solve(const std::vector<double>& nums) {
    if (nums.size() == 1) { return std::abs(nums[0] - 24) < 1e-9; } // goal
    for (std::size_t i = 0; i < nums.size(); ++i) {
        for (std::size_t j = i + 1; j < nums.size(); ++j) {    // unordered pair
            double a = nums[i], b = nums[j];
            // ... what do we do with a and b?
        }
    }
}
```

Step 3: Keep the Other Numbers

Question. We merge $a =$ `nums[i]` and $b =$ `nums[j]` into one result. Let `rest` be the numbers that stay on the table (the result will be added to `rest` later). Try `nums = {3, 3, 8, 8}` with $i = 0$, $j = 2$ (so $a = 3$, $b = 8$). What should `rest` contain, and how do we build it?

(A) All of `nums`, unchanged.

(B) Only `{a, b}`, the pair we picked.

(C) Every `nums[t]` with `nums[t] != a && nums[t] != b`.

(D) Every `nums[t]` with `t != i && t != j`.

```
bool solve(const std::vector<double>& nums) {
    if (nums.size() == 1) { return std::abs(nums[0] - 24) < 1e-9; } // goal
    for (std::size_t i = 0; i < nums.size(); ++i) {
        for (std::size_t j = i + 1; j < nums.size(); ++j) {    // unordered pair
            double a = nums[i], b = nums[j];
            // ???
        }
    }
}
```

Step 3: Keep the Other Numbers

Question. We merge $a =$ `nums[i]` and $b =$ `nums[j]` into one result. Let `rest` be the numbers that stay on the table (the result will be added to `rest` later). Try `nums = {3, 3, 8, 8}` with $i = 0$, $j = 2$ (so $a = 3$, $b = 8$). What should `rest` contain, and how do we build it?

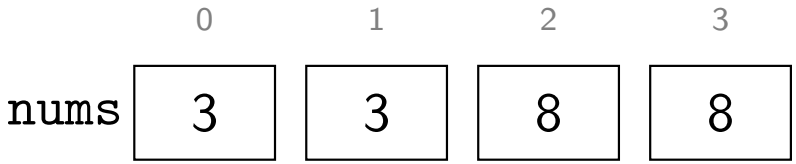
(A) All of `nums`, unchanged.
(B) Only `{a, b}`, the pair we picked.
(C) Every `nums[t]` with `nums[t] != a && nums[t] != b`.
(D) **Every `nums[t]` with `t != i && t != j`.**

(D), giving $\{3, 8\}$. Skip by **position**, not by value. (C) compares values, so it throws away *all* the 3s and *all* the 8s and leaves `rest` empty. (A) would count a and b twice. (B) keeps exactly what we want to remove.

```
bool solve(const std::vector<double>& nums) {
    if (nums.size() == 1) { return std::abs(nums[0] - 24) < 1e-9; } // goal
    for (std::size_t i = 0; i < nums.size(); ++i) {
        for (std::size_t j = i + 1; j < nums.size(); ++j) {    // unordered pair
            double a = nums[i], b = nums[j];
            std::vector<double> rest;                          // the other numbers
            for (std::size_t t = 0; t < nums.size(); ++t) {
                if (t != i && t != j) { rest.push_back(nums[t]); }
            }
            // ... what can we do with a and b?
        }
    }
}
```

Trace: Building `rest`, One Position at a Time

`nums = {3, 3, 8, 8}`, $i = 0$, $j = 2$, so $a = 3$ and $b = 8$. The loop over `t` copies every position except `i` and `j`.

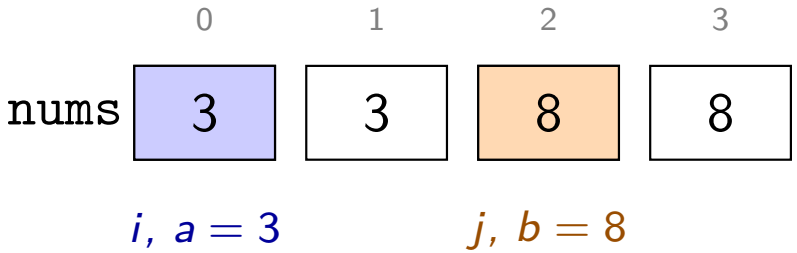


`rest` *(empty)*

Positions: `t = 0, 1, 2, 3`. Start with an empty `rest`.

Trace: Building `rest`, One Position at a Time

`nums = {3, 3, 8, 8}`, $i = 0$, $j = 2$, so $a = 3$ and $b = 8$. The loop over `t` copies every position except `i` and `j`.

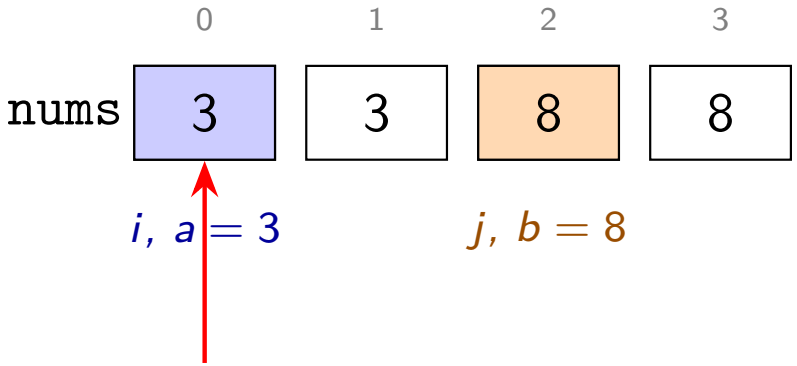


`rest` *(empty)*

Mark the pair: `nums[i]` is a, `nums[j]` is b.

Trace: Building `rest`, One Position at a Time

`nums = {3, 3, 8, 8}`, $i = 0$, $j = 2$, so $a = 3$ and $b = 8$. The loop over `t` copies every position except `i` and `j`.

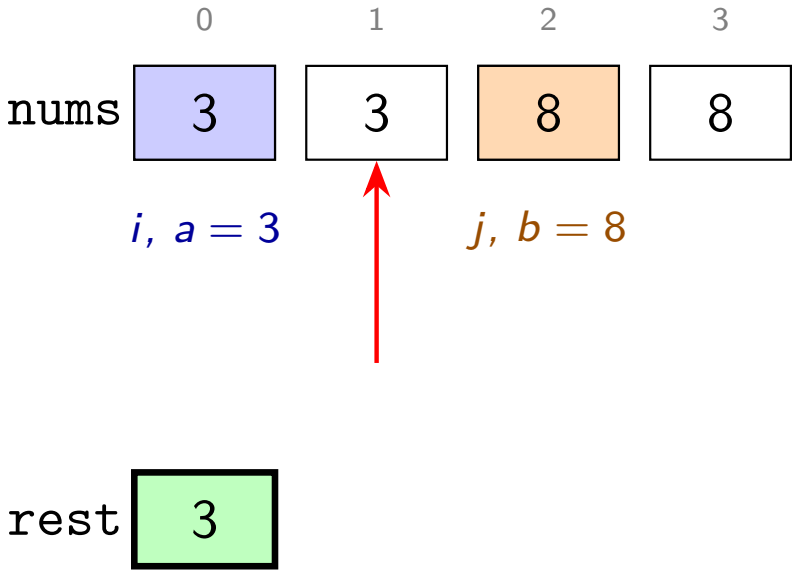


`rest` *(empty)*

`t = 0`: `t == i`, so **skip**.

Trace: Building `rest`, One Position at a Time

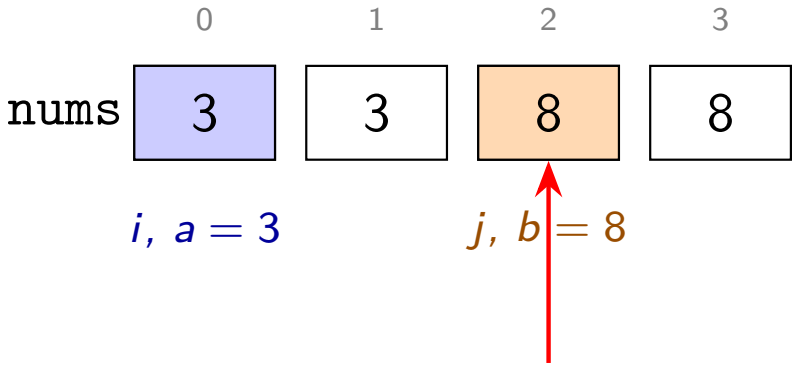
`nums = {3, 3, 8, 8}`, $i = 0$, $j = 2$, so $a = 3$ and $b = 8$. The loop over `t` copies every position except `i` and `j`.

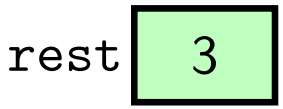


`t = 1`: not `i`, not `j`, so `rest.push_back(nums[1])`: push 3.

Trace: Building `rest`, One Position at a Time

`nums = {3, 3, 8, 8}`, $i = 0$, $j = 2$, so $a = 3$ and $b = 8$. The loop over `t` copies every position except `i` and `j`.

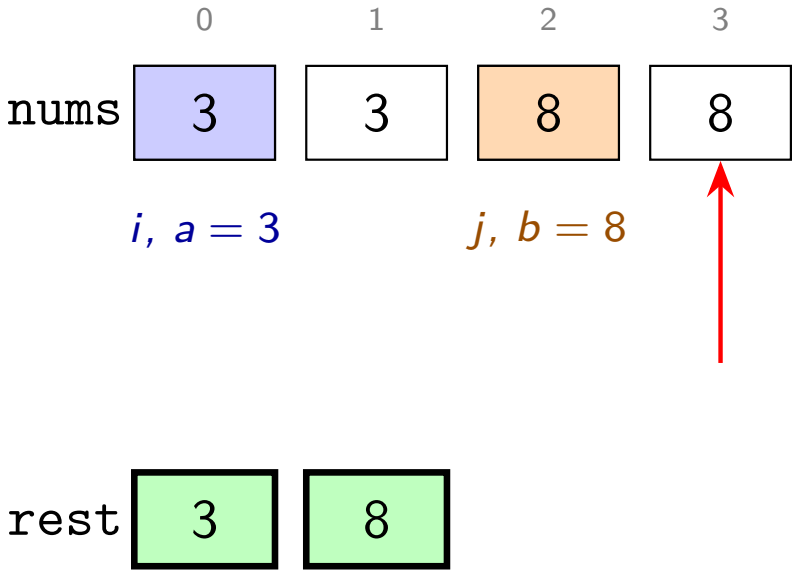




`t = 2`: `t == j`, so **skip**.

Trace: Building `rest`, One Position at a Time

`nums = {3, 3, 8, 8}`, $i = 0$, $j = 2$, so $a = 3$ and $b = 8$. The loop over `t` copies every position except `i` and `j`.



`t = 3`: push 8. Done: `rest = {3, 8}`, the numbers that stay on the table.

Step 4: The Results of a Pair

Question. Try it with $a = 6$, $b = 0$. Which results can you write down? Then decide which results the code should list for a general pair a, b.

(A) `a+b, a*b, a-b, a/b`

(B) `a+b, a*b, a-b, b-a`; add `a/b` only if `b` is not 0; add `b/a` only if `a` is not 0

(C) All six every time, including `a/b` and `b/a`

(D) Only `a+b` and `a*b`

```
bool solve(const std::vector<double>& nums) {
    if (nums.size() == 1) { return std::abs(nums[0] - 24) < 1e-9; } // goal
    for (std::size_t i = 0; i < nums.size(); ++i) {
        for (std::size_t j = i + 1; j < nums.size(); ++j) {    // unordered pair
            double a = nums[i], b = nums[j];
            std::vector<double> rest;                          // the other numbers
            for (std::size_t t = 0; t < nums.size(); ++t) {
                if (t != i && t != j) { rest.push_back(nums[t]); }
            }
            // ??? list the results of a and b
        }
    }
}
```

Step 4: The Results of a Pair

Question. Try it with $a = 6$, $b = 0$. Which results can you write down? Then decide which results the code should list for a general pair a, b.

(A) `a+b, a*b, a-b, a/b`

(B) `a+b, a*b, a-b, b-a`; **add `a/b` only if `b` is not 0; add `b/a` only if `a` is not 0**

(C) All six every time, including a/b and b/a

(D) Only a+b and a*b

(B). For $a = 6$, $b = 0$: $6 + 0$, 6×0, $6 - 0$, $0 - 6$, $0 \div 6$ are fine, but $6 \div 0$ is not. Subtraction and division are **not symmetric**, so we need both orders. (A) misses $b - a$, $b \div a$. (C) divides by zero. (D) loses most results. Use a tolerance: `std::abs(b) > 1e-9`.

```
bool solve(const std::vector<double>& nums) {
    if (nums.size() == 1) { return std::abs(nums[0] - 24) < 1e-9; } // goal
    for (std::size_t i = 0; i < nums.size(); ++i) {
        for (std::size_t j = i + 1; j < nums.size(); ++j) {    // unordered pair
            double a = nums[i], b = nums[j];
            std::vector<double> rest;                          // the other numbers
            for (std::size_t t = 0; t < nums.size(); ++t) {
                if (t != i && t != j) { rest.push_back(nums[t]); }
            }
            std::vector<double> results = {a + b, a * b, a - b, b - a};
            if (std::abs(b) > 1e-9) { results.push_back(a / b); } // skip 0 or near 0
            if (std::abs(a) > 1e-9) { results.push_back(b / a); }
            // ... now try each result
        }
    }
}
```

Trace: Building `results`, One Result at a Time

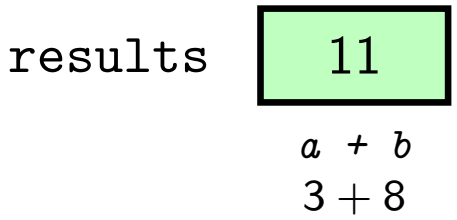
Same pair: $a = 3$, $b = 8$, `rest = {3, 8}`. The vector `results` holds every number the pair can turn into.

`results`

The first four results are always allowed. Division is added only if the divisor is not 0.

Trace: Building `results`, One Result at a Time

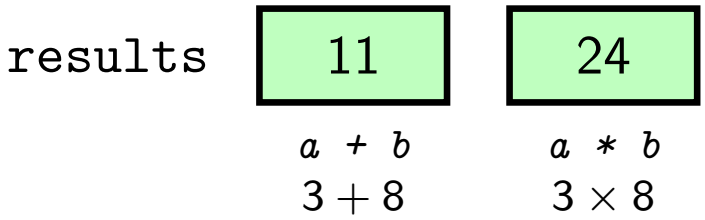
Same pair: $a = 3$, $b = 8$, `rest = {3, 8}`. The vector `results` holds every number the pair can turn into.



`results = {a + b, ...}`: $3 + 8 = 11$.

Trace: Building `results`, One Result at a Time

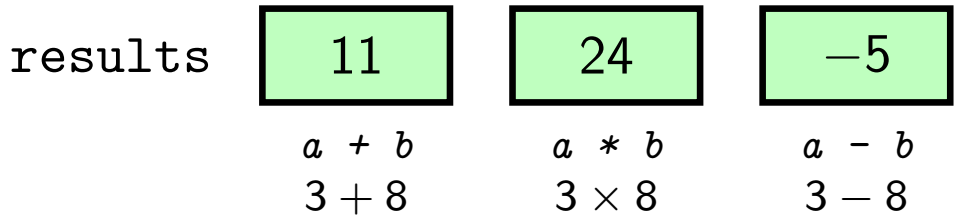
Same pair: $a = 3$, $b = 8$, `rest = {3, 8}`. The vector `results` holds every number the pair can turn into.



Next `a * b`: $3 \times 8 = 24$. This one is already 24, but the search will not look at it until all of `rest` is combined with it.

Trace: Building `results`, One Result at a Time

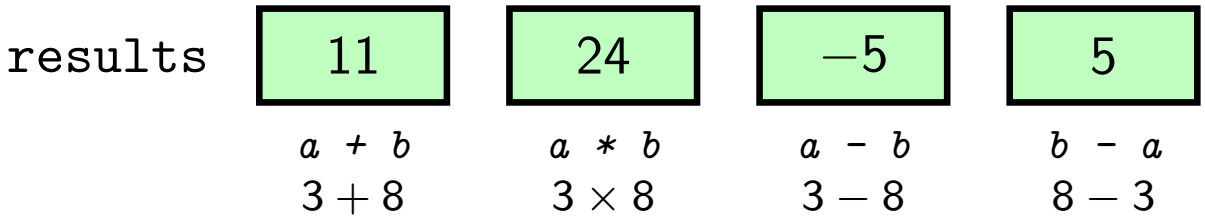
Same pair: $a = 3$, $b = 8$, `rest = {3, 8}`. The vector `results` holds every number the pair can turn into.



Next `a - b`: $3 - 8 = -5$.

Trace: Building `results`, One Result at a Time

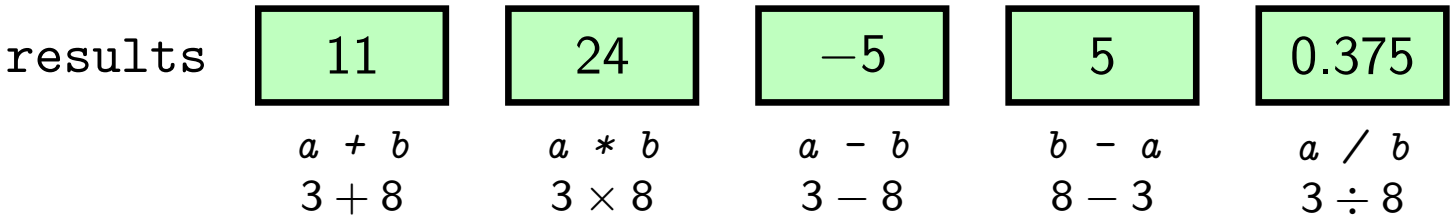
Same pair: $a = 3$, $b = 8$, `rest = {3, 8}`. The vector `results` holds every number the pair can turn into.



Next `b - a`: $8 - 3 = 5$. Subtraction is not symmetric, so both orders are needed.

Trace: Building `results`, One Result at a Time

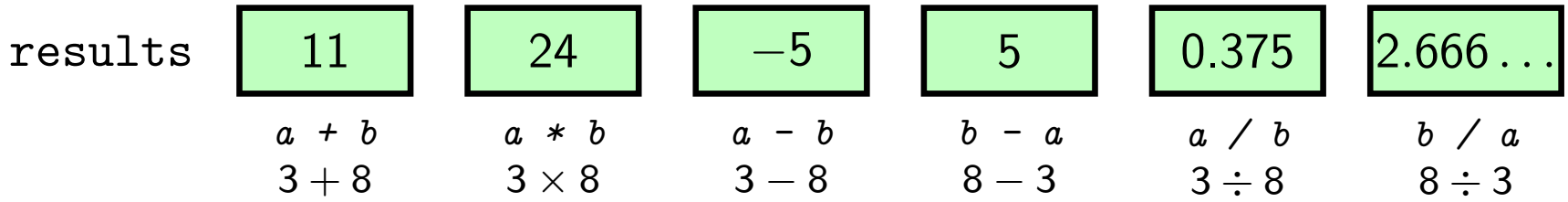
Same pair: $a = 3$, $b = 8$, `rest = {3, 8}`. The vector `results` holds every number the pair can turn into.



`if (std::abs(b) > 1e-9)`: $b = 8$ is not 0, so add `a / b` $= 0.375$.

Trace: Building `results`, One Result at a Time

Same pair: $a = 3$, $b = 8$, `rest = {3, 8}`. The vector `results` holds every number the pair can turn into.



`if (std::abs(a) > 1e-9)`: $a = 3$ is not 0, so add `b / a` $= 2.666\ldots$ (the `double` keeps about 16 digits; we only print a few). Six results. (If b were 0, `a / b` would not be added.)

Step 5: Try Each Result, Then Undo

Question. For each result `r`: build the smaller problem (`rest` plus `r`), test it, and restore `rest`. Which order is correct?

(A) `rest.pop_back(); rest.push_back(r); if (solve(rest)) return true;`

(B) `rest.push_back(r); rest.pop_back(); if (solve(rest)) return true;`

(C) `rest.push_back(r); if (solve(rest)) return true; rest.pop_back();`

(D) `rest.push_back(r); if (solve(rest)) return true;` (no undo)

```
bool solve(const std::vector<double>& nums) {
    if (nums.size() == 1) { return std::abs(nums[0] - 24) < 1e-9; } // goal
    for (std::size_t i = 0; i < nums.size(); ++i) {
        for (std::size_t j = i + 1; j < nums.size(); ++j) {    // unordered pair
            double a = nums[i], b = nums[j];
            // rest and results are built as in Steps 3 and 4
            // ??? try each result r
        }
    }
}
```

Step 5: Try Each Result, Then Undo

Question. For each result `r`: build the smaller problem (`rest` plus `r`), test it, and restore `rest`. Which order is correct?

(A) `rest.pop_back(); rest.push_back(r); if (solve(rest)) return true;`

(B) `rest.push_back(r); rest.pop_back(); if (solve(rest)) return true;`

(C) `rest.push_back(r); if (solve(rest)) return true; rest.pop_back();`

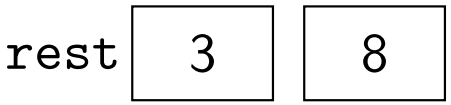
(D) `rest.push_back(r); if (solve(rest)) return true;` (no undo)

(C). Push, test, pop: afterwards `rest` is again the other numbers. (A) pops an empty vector: undefined behavior. (B) tests `rest` without `r`. (D) never undoes, so `rest` grows to `{others, r1, r2}` and the next call solves the wrong problem.

```
bool solve(const std::vector<double>& nums) {
    if (nums.size() == 1) { return std::abs(nums[0] - 24) < 1e-9; } // goal
    for (std::size_t i = 0; i < nums.size(); ++i) {
        for (std::size_t j = i + 1; j < nums.size(); ++j) {    // unordered pair
            double a = nums[i], b = nums[j];
            // rest and results are built as in Steps 3 and 4
            for (double r : results) {
                rest.push_back(r);                             // choose: pair -> r
                if (solve(rest)) { return true; }              // explore, stop early
                rest.pop_back();                               // undo
            }
        }
    }
    return false;                // every pair and every result failed
}
```

Trace: Push, Test, Pop on `rest`

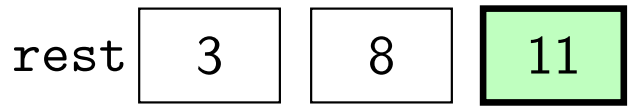
`rest = {3, 8}`, and the first two results are `r = 11`, then `r = 24`. Watch `rest` change.



Start of the loop: `rest` has $n - 2 = 2$ numbers.

Trace: Push, Test, Pop on `rest`

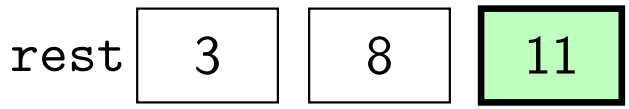
`rest = {3, 8}`, and the first two results are `r = 11`, then `r = 24`. Watch `rest` change.



`r = 11`: `rest.push_back(r)`. Now `rest = {3, 8, 11}`: three numbers, the smaller problem.

Trace: Push, Test, Pop on `rest`

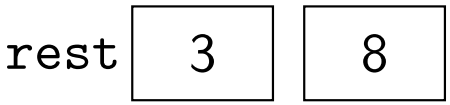
`rest = {3, 8}`, and the first two results are `r = 11`, then `r = 24`. Watch `rest` change.



`solve({3, 8, 11})` returns `false`, so we do not return.

Trace: Push, Test, Pop on `rest`

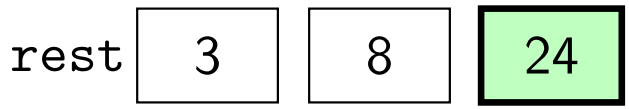
`rest = {3, 8}`, and the first two results are `r = 11`, then `r = 24`. Watch `rest` change.



`rest.pop_back()`: `rest = {3, 8}` again, ready for the next result.

Trace: Push, Test, Pop on `rest`

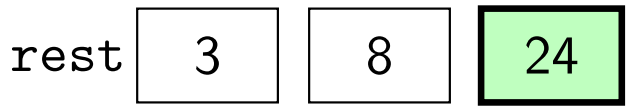
`rest = {3, 8}`, and the first two results are `r = 11`, then `r = 24`. Watch `rest` change.



`r = 24`: push. `rest = {3, 8, 24}`.

Trace: Push, Test, Pop on `rest`

`rest = {3, 8}`, and the first two results are `r = 11`, then `r = 24`. Watch `rest` change.



`solve({3, 8, 24})` also returns `false` (the other pair choices and results are tried by other iterations), so pop again: `rest = {3, 8}`.

Make 24: The Whole Function

The five steps together: a move **picks a pair** $\{a, b\}$ and replaces it by one result, so n numbers become $n - 1$.

```
bool solve(const std::vector<double>& nums) {
    if (nums.size() == 1) { return std::abs(nums[0] - 24) < 1e-9; } // goal
    for (std::size_t i = 0; i < nums.size(); ++i) {
        for (std::size_t j = i + 1; j < nums.size(); ++j) {    // unordered pair
            double a = nums[i], b = nums[j];
            std::vector<double> rest;                          // the other numbers
            for (std::size_t t = 0; t < nums.size(); ++t) {
                if (t != i && t != j) { rest.push_back(nums[t]); }
            }
            std::vector<double> results = {a + b, a * b, a - b, b - a};
            if (std::abs(b) > 1e-9) { results.push_back(a / b); } // skip 0 or near 0
            if (std::abs(a) > 1e-9) { results.push_back(b / a); }
            for (double r : results) {
                rest.push_back(r);                             // choose: pair -> r
                if (solve(rest)) { return true; }              // explore, stop early
                rest.pop_back();                               // undo
            }
        }
    }
    return false;
}
```

Version 1: Ordered Pairs, 4 Operations

The numbers are (a, b, c, d). Version 1 picks an **ordered** pair (x, y): x is any number, and y is one of its **friends**, meaning any *other* number. Then it tries $x + y$, $x - y$, $x \times y$, $x \div y$.

(a, b)	(a, c)	(a, d)
(b, a)	(b, c)	(b, d)
(c, a)	(c, b)	(c, d)
(d, a)	(d, b)	(d, c)

12 ordered pairs $= 4 \times 3$

Numbers left	**Numbers $\times$ friends**	**Ops**	**Moves**
4 (first moves)	$4 \times 3 = 12$	4	?
3 left	$3 \times (3 - 1) = 6$	4	?
2 left	$2 \times (2 - 1) = 2$	4	?
Complete sequences			?

Question: how many first moves? 4 numbers, and each number has 3 friends, so $4 \times 3 = 12$ ordered pairs. The 4 in "$\times 4$" counts operations, not numbers.

Version 1: Ordered Pairs, 4 Operations

The numbers are (a, b, c, d). Version 1 picks an **ordered** pair (x, y): x is any number, and y is one of its **friends**, meaning any *other* number. Then it tries $x + y$, $x - y$, $x \times y$, $x \div y$.

(a, b)	(a, c)	(a, d)
(b, a)	(b, c)	(b, d)
(c, a)	(c, b)	(c, d)
(d, a)	(d, b)	(d, c)

12 ordered pairs $= 4 \times 3$

Numbers left	**Numbers $\times$ friends**	**Ops**	**Moves**
4 (first moves)	$4 \times 3 = 12$	4	$12 \times 4 =$ **48**
3 left	$3 \times (3 - 1) = 6$	4	?
2 left	$2 \times (2 - 1) = 2$	4	?
Complete sequences			?

12 ordered pairs, each with 4 operations. Next: after the first move there are 3 numbers left, and each has $3 - 1 = 2$ friends.

Version 1: Ordered Pairs, 4 Operations

The numbers are (a, b, c, d). Version 1 picks an **ordered** pair (x, y): x is any number, and y is one of its **friends**, meaning any *other* number. Then it tries $x + y$, $x - y$, $x \times y$, $x \div y$.

(a, b)	(a, c)	(a, d)
(b, a)	(b, c)	(b, d)
(c, a)	(c, b)	(c, d)
(d, a)	(d, b)	(d, c)

12 ordered pairs $= 4 \times 3$

Numbers left	**Numbers $\times$ friends**	**Ops**	**Moves**
4 (first moves)	$4 \times 3 = 12$	4	$12 \times 4 =$ **48**
3 left	$3 \times (3 - 1) = 6$	4	$6 \times 4 =$ **24**
2 left	$2 \times (2 - 1) = 2$	4	?
Complete sequences			?

Version 1: Ordered Pairs, 4 Operations

The numbers are (a, b, c, d). Version 1 picks an **ordered** pair (x, y): x is any number, and y is one of its **friends**, meaning any *other* number. Then it tries $x + y$, $x - y$, $x \times y$, $x \div y$.

(a, b)	(a, c)	(a, d)
(b, a)	(b, c)	(b, d)
(c, a)	(c, b)	(c, d)
(d, a)	(d, b)	(d, c)

12 ordered pairs $= 4 \times 3$

Numbers left	**Numbers $\times$ friends**	**Ops**	**Moves**
4 (first moves)	$4 \times 3 = 12$	4	$12 \times 4 = \mathbf{48}$
3 left	$3 \times (3 - 1) = 6$	4	$6 \times 4 = \mathbf{24}$
2 left	$2 \times (2 - 1) = 2$	4	$2 \times 4 = \mathbf{8}$
Complete sequences			?

Version 1: Ordered Pairs, 4 Operations

The numbers are (a, b, c, d). Version 1 picks an **ordered** pair (x, y): x is any number, and y is one of its **friends**, meaning any *other* number. Then it tries $x + y$, $x - y$, $x \times y$, $x \div y$.

(a, b)	(a, c)	(a, d)
(b, a)	(b, c)	(b, d)
(c, a)	(c, b)	(c, d)
(d, a)	(d, b)	(d, c)

12 ordered pairs $= 4 \times 3$

Numbers left	**Numbers $\times$ friends**	**Ops**	**Moves**
4 (first moves)	$4 \times 3 = 12$	4	$12 \times 4 = \mathbf{48}$
3 left	$3 \times (3 - 1) = 6$	4	$6 \times 4 = \mathbf{24}$
2 left	$2 \times (2 - 1) = 2$	4	$2 \times 4 = \mathbf{8}$
Complete sequences			$48 \times 24 \times 8 = \mathbf{9216}$

Version 1 makes **9216** complete sequences. (Theoretical count: every operation allowed.)

Version 2: Unordered Pairs, 6 Operations

Version 2 picks an **unordered** pair $\{x, y\}$: friendship is mutual, so each pair was counted twice and we **halve**. It lists **6** operations: $x + y,\ x \times y,\ x - y,\ y - x,\ x \div y,\ y \div x$.

$\{a, b\}$ $\{a, c\}$ $\{a, d\}$
$\{b, c\}$ $\{b, d\}$ $\{c, d\}$ **6 unordered pairs** $= \binom{4}{2}$

Numbers left	**Pairs**	**Ops**	**Moves**
4 (first moves)	$4 \times 3 \div 2 = 6$	6	?
3 left	$3 \times 2 \div 2 = 3$	6	?
2 left	$2 \times 1 \div 2 = 1$	6	?
Complete sequences			?

Half as many pairs, but 6 operations instead of 4. Fill in the table.

Version 2: Unordered Pairs, 6 Operations

Version 2 picks an **unordered** pair $\{x, y\}$: friendship is mutual, so each pair was counted twice and we **halve**. It lists **6** operations: $x + y,\ x \times y,\ x - y,\ y - x,\ x \div y,\ y \div x$.

$\{a, b\}$ $\{a, c\}$ $\{a, d\}$
$\{b, c\}$ $\{b, d\}$ $\{c, d\}$ **6 unordered pairs** $= \binom{4}{2}$

Numbers left	**Pairs**	**Ops**	**Moves**
4 (first moves)	$4 \times 3 \div 2 = 6$	6	$6 \times 6 = \mathbf{36}$
3 left	$3 \times 2 \div 2 = 3$	6	?
2 left	$2 \times 1 \div 2 = 1$	6	?
Complete sequences			?

Version 2: Unordered Pairs, 6 Operations

Version 2 picks an **unordered** pair $\{x, y\}$: friendship is mutual, so each pair was counted twice and we **halve**. It lists **6** operations: $x + y,\ x \times y,\ x - y,\ y - x,\ x \div y,\ y \div x$.

$\{a, b\}$ $\{a, c\}$ $\{a, d\}$
$\{b, c\}$ $\{b, d\}$ $\{c, d\}$ **6 unordered pairs** $= \binom{4}{2}$

Numbers left	**Pairs**	**Ops**	**Moves**
4 (first moves)	$4 \times 3 \div 2 = 6$	6	$6 \times 6 = \mathbf{36}$
3 left	$3 \times 2 \div 2 = 3$	6	$3 \times 6 = \mathbf{18}$
2 left	$2 \times 1 \div 2 = 1$	6	?
Complete sequences			?

Version 2: Unordered Pairs, 6 Operations

Version 2 picks an **unordered** pair $\{x, y\}$: friendship is mutual, so each pair was counted twice and we **halve**. It lists **6** operations: $x + y,\ x \times y,\ x - y,\ y - x,\ x \div y,\ y \div x$.

$\{a, b\}$ $\{a, c\}$ $\{a, d\}$
$\{b, c\}$ $\{b, d\}$ $\{c, d\}$ **6 unordered pairs** $= \binom{4}{2}$

Numbers left	**Pairs**	**Ops**	**Moves**
4 (first moves)	$4 \times 3 \div 2 = 6$	6	$6 \times 6 = \mathbf{36}$
3 left	$3 \times 2 \div 2 = 3$	6	$3 \times 6 = \mathbf{18}$
2 left	$2 \times 1 \div 2 = 1$	6	$1 \times 6 = \mathbf{6}$
Complete sequences			?

Version 2: Unordered Pairs, 6 Operations

Version 2 picks an **unordered** pair $\{x, y\}$: friendship is mutual, so each pair was counted twice and we **halve**. It lists **6** operations: $x + y,\ x \times y,\ x - y,\ y - x,\ x \div y,\ y \div x$.

$\{a, b\}$ $\{a, c\}$ $\{a, d\}$
$\{b, c\}$ $\{b, d\}$ $\{c, d\}$ **6 unordered pairs** $= \binom{4}{2}$

Numbers left	**Pairs**	**Ops**	**Moves**
4 (first moves)	$4 \times 3 \div 2 = 6$	6	$6 \times 6 = \mathbf{36}$
3 left	$3 \times 2 \div 2 = 3$	6	$3 \times 6 = \mathbf{18}$
2 left	$2 \times 1 \div 2 = 1$	6	$1 \times 6 = \mathbf{6}$
Complete sequences			$36 \times 18 \times 6 = \mathbf{3888}$

Version 1: 9216. **Version 2:** 3888, about 2.4× fewer.

Version 2: Unordered Pairs, 6 Operations

Version 2 picks an **unordered** pair $\{x, y\}$: friendship is mutual, so each pair was counted twice and we **halve**. It lists **6** operations: $x + y,\ x \times y,\ x - y,\ y - x,\ x \div y,\ y \div x$.

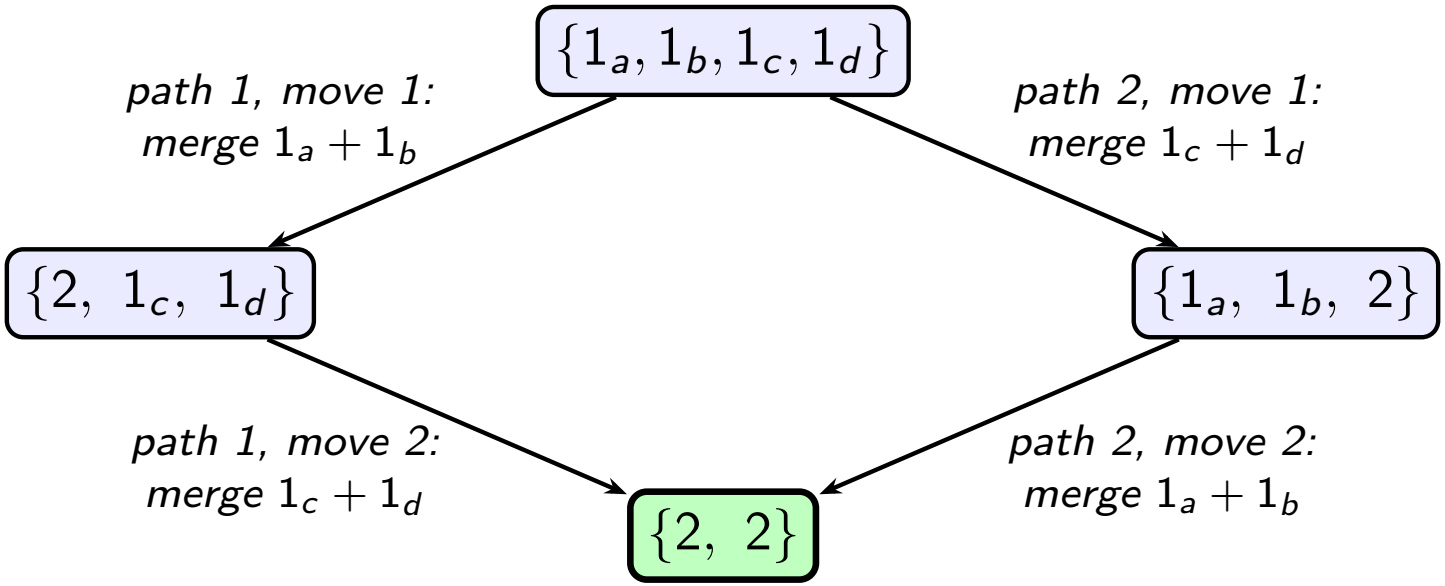
$\{a, b\}$ $\{a, c\}$ $\{a, d\}$
$\{b, c\}$ $\{b, d\}$ $\{c, d\}$ **6 unordered pairs** $= \binom{4}{2}$

Numbers left	**Pairs**	**Ops**	**Moves**
4 (first moves)	$4 \times 3 \div 2 = 6$	6	$6 \times 6 = \mathbf{36}$
3 left	$3 \times 2 \div 2 = 3$	6	$3 \times 6 = \mathbf{18}$
2 left	$2 \times 1 \div 2 = 1$	6	$1 \times 6 = \mathbf{6}$
Complete sequences			$36 \times 18 \times 6 = \mathbf{3888}$

Why 6 operations on unordered pairs is better. Version 1 tries $x + y = y + x$ and $x \times y = y \times x$ twice for nothing. Only $-$ and $\div$ depend on order, so Version 2 keeps both orders for those two only: same numbers reachable, fewer moves.

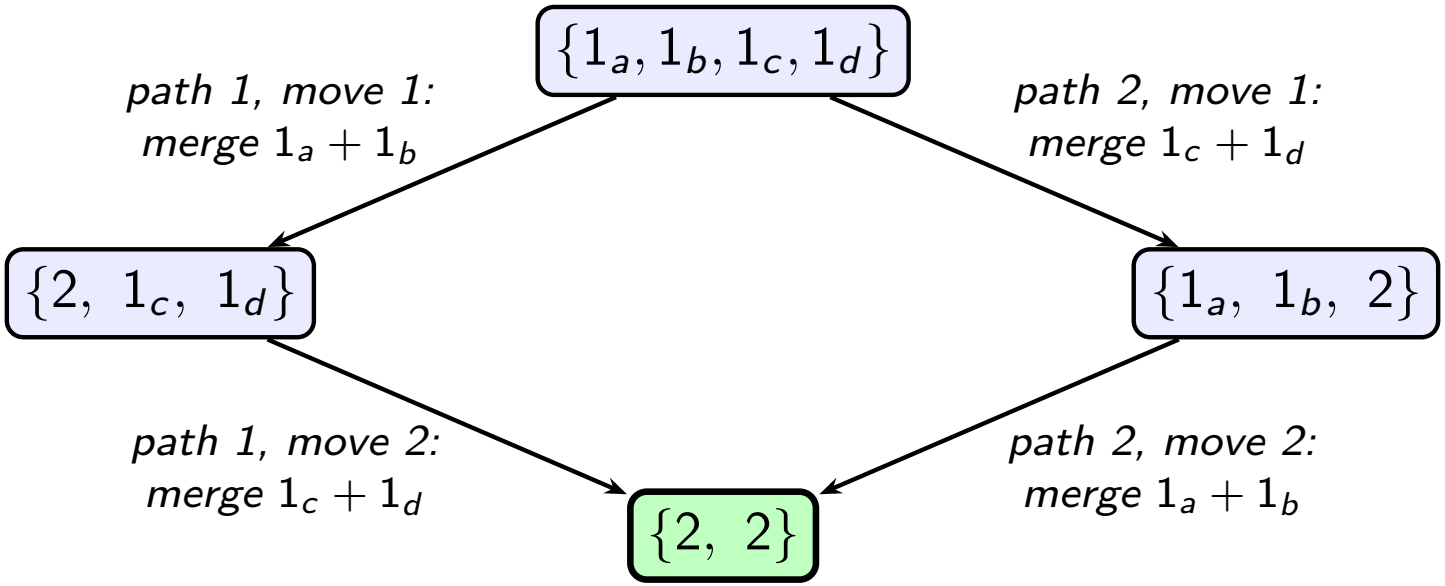
Duplicate States: Two Paths to the Same Place

Four ones are on the table. Give them names so we can tell them apart: $\{1_a,\ 1_b,\ 1_c,\ 1_d\}$. Two moves reach $\{2, 2\}$: merge one pair, then the other pair. Two move orders do it:



Duplicate States: Two Paths to the Same Place

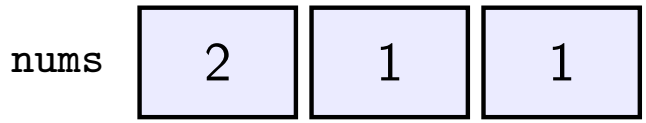
Four ones are on the table. Give them names so we can tell them apart: $\{1_a,\ 1_b,\ 1_c,\ 1_d\}$. Two moves reach $\{2, 2\}$: merge one pair, then the other pair. Two move orders do it:



Same two merges, opposite order, so the search below `{2, 2}` is **identical** both times. Remember the **state** (the sorted multiset of numbers). If seen before, return `false`: the earlier visit already explored it, and if it had found 24 we would have stopped.

Building the Key: a `vector<long long>`

A state is the set of numbers on the table. We turn it into a **key**: a row of whole-number boxes, built one number at a time.

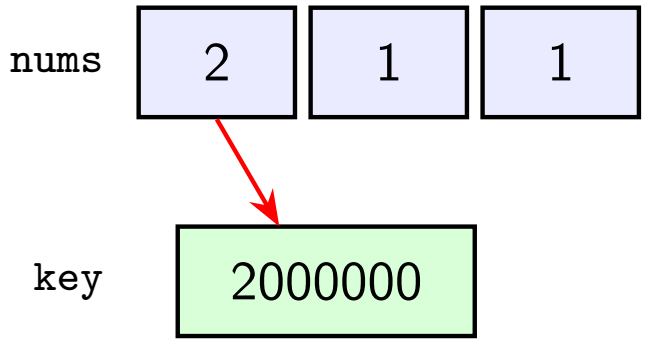


`key` *(empty)*

`nums = {2, 1, 1}`. Start with an empty `key`.

Building the Key: a `vector<long long>`

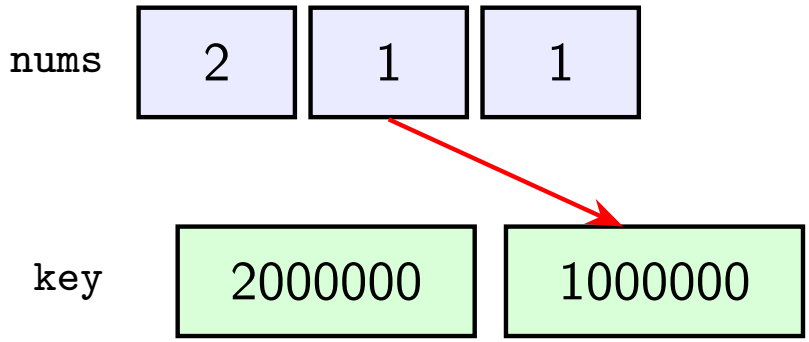
A state is the set of numbers on the table. We turn it into a **key**: a row of whole-number boxes, built one number at a time.



`x = 2`: `llround(2 * 1e6)` = 2000000, so `push_back(2000000)`.

Building the Key: a `vector<long long>`

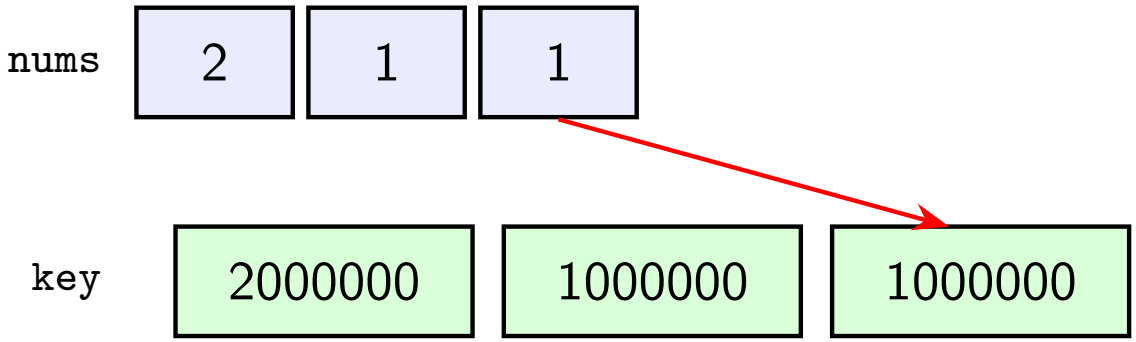
A state is the set of numbers on the table. We turn it into a **key**: a row of whole-number boxes, built one number at a time.



`x = 1`: $1 \times 10^6 = 1000000$ goes into the next box.

Building the Key: a `vector<long long>`

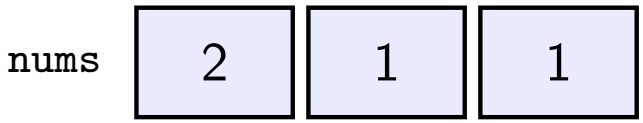
A state is the set of numbers on the table. We turn it into a **key**: a row of whole-number boxes, built one number at a time.

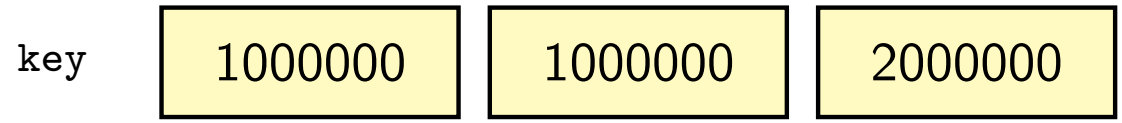


`x = 1`: another 1000000. The key has 3 boxes, one for each number.

Building the Key: a `vector<long long>`

A state is the set of numbers on the table. We turn it into a **key**: a row of whole-number boxes, built one number at a time.

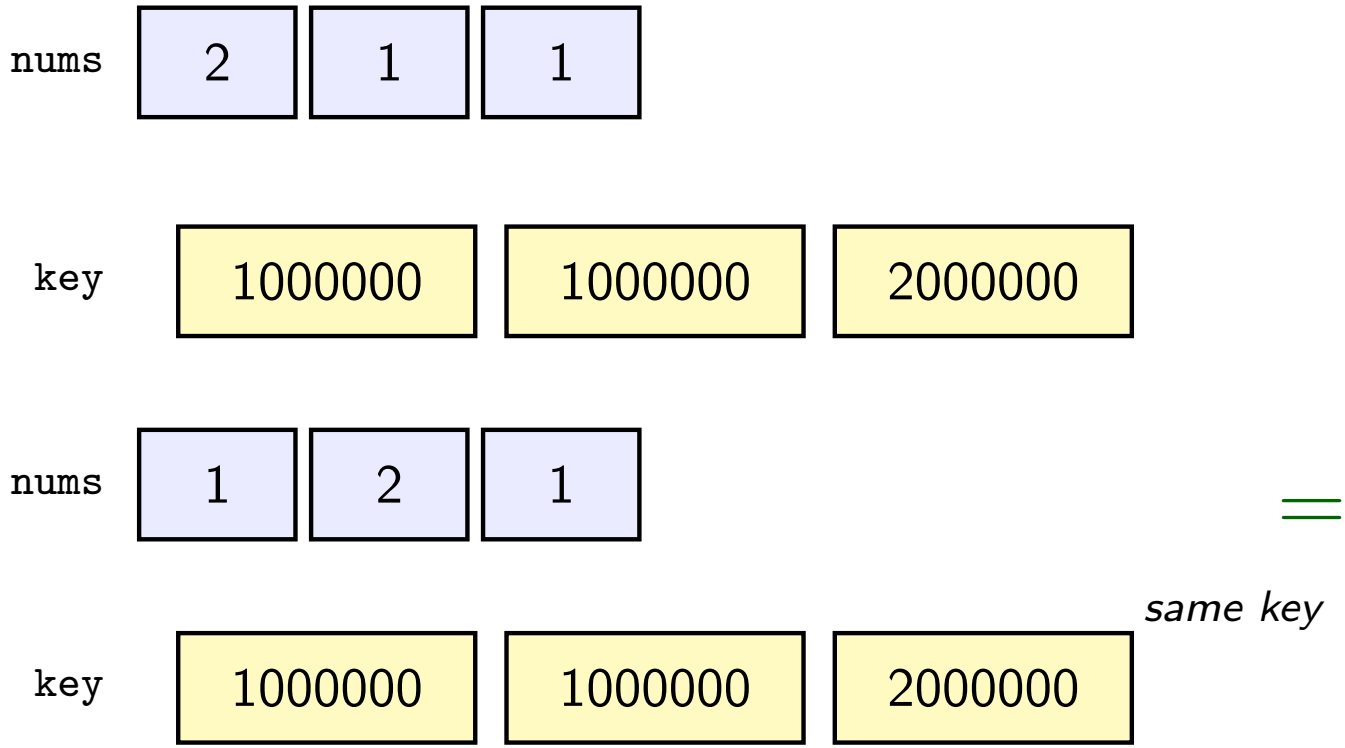




`std::sort(key.begin(), key.end())`: smallest first. Now the order the numbers arrived in no longer matters.

Building the Key: a `vector<long long>`

A state is the set of numbers on the table. We turn it into a **key**: a row of whole-number boxes, built one number at a time.



The same numbers in another order, `{1, 2, 1}`, give the **same sorted key**. Same key means same state: keep one copy.

Remembering States: A `std::set` of Keys

Illustrative fragment: the pair loops are omitted.

```
using State = std::vector<long long>;  // sorted, rounded: approximate!

bool solve(const std::vector<double>& nums, std::set<State>& seen) {
    if (nums.size() == 1) { return std::abs(nums[0] - 24) < 1e-9; }
    State key;                                  // the state of the search
    for (double x : nums) { key.push_back(std::llround(x * 1e6)); }
    std::sort(key.begin(), key.end());
    if (!seen.insert(key).second) { return false; } // seen before
    // ... the same pair loops as before, calling solve(rest, seen) ...
}
```

- `std::set` here just *remembers* states. `insert(key).second` is `true` if the key was new, `false` if it was already there.
- `return false`: **this state was already searched and had no solution** (else we would have stopped). Reuse that answer.
- **Sort** the key, because $\{2, 1, 1\}$ and $\{1, 2, 1\}$ are the same state.
- **Caution:** a rounded key is only **approximate**: different states can share a key (a solution may be skipped), and equal states can miss each other (lost savings only).

Measured: Make 24 With `{1, 1, 1, 1}`

$\{1, 1, 1, 1\}$ has **no solution**, so the search must visit everything it cannot cut. Counting calls of `solve`:

Version	**Calls**
Ordered pairs, 4 operations	9 841
Unordered pairs, 6 operations (symmetry) + remember seen states	

Measured: Make 24 With `{1, 1, 1, 1}`

$\{1, 1, 1, 1\}$ has **no solution**, so the search must visit everything it cannot cut. Counting calls of `solve`:

Version	**Calls**
Ordered pairs, 4 operations	9 841
Unordered pairs, 6 operations (symmetry)	4 141
+ remember seen states	**139**

139 calls are made, but only **13 different states** are expanded: a call that reaches an already-seen state returns immediately. For $\{3, 3, 8, 8\}$, which *does* have a solution ($8 \div (3 - 8 \div 3) = 24$), the same two versions make 1 401 and 749 calls.

Duplicate Letters: Skip the Repeated Branch

On Class 15's slip, `printAll("ABB", 0)` printed 6 lines for 3 different strings. At each **seat**, trying the same *letter* twice repeats the whole branch.

```
void printUnique(std::string& s, int k) {
    if (k == static_cast<int>(s.size())) { std::cout << s << "\n";
        return; }
    std::set<char> tried;                          // letters already
        tried at this seat
    for (int i = k; i < static_cast<int>(s.size()); ++i) {
        if (!tried.insert(s[i]).second) { continue; } // same letter
            again: skip this branch
        std::swap(s[k], s[i]);                     // choose
        printUnique(s, k + 1);                     // explore
        std::swap(s[k], s[i]);                     // undo
    }
}
```

Duplicate Letters: Skip the Repeated Branch

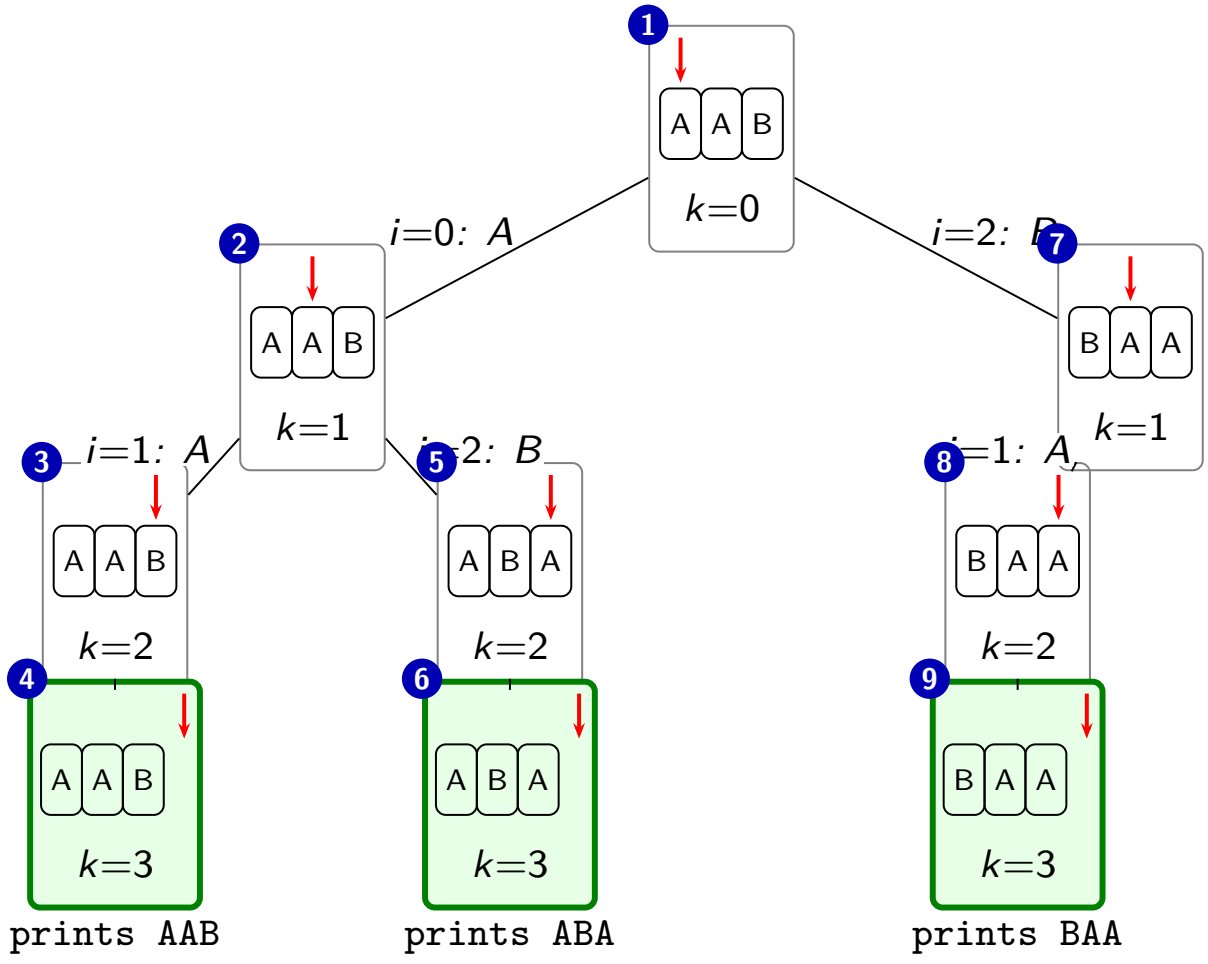
On Class 15's slip, `printAll("ABB", 0)` printed 6 lines for 3 different strings. At each **seat**, trying the same *letter* twice repeats the whole branch.

```
void printUnique(std::string& s, int k) {
    if (k == static_cast<int>(s.size())) { std::cout << s << "\n";
        return; }
    std::set<char> tried;                          // letters already
        tried at this seat
    for (int i = k; i < static_cast<int>(s.size()); ++i) {
        if (!tried.insert(s[i]).second) { continue; } // same letter
            again: skip this branch
        std::swap(s[k], s[i]);                     // choose
        printUnique(s, k + 1);                     // explore
        std::swap(s[k], s[i]);                     // undo
    }
}
```

`"AAB"` prints `AAB`, `ABA`, `BAA`: $3!/2! = 3$ strings, and the number of calls drops from **16** to **9**. For `"ABC"` nothing is skipped (16 calls). The `tried` set belongs to **one call**: a new set at every seat.

The Call Tree of `printUnique("AAB", 0)`

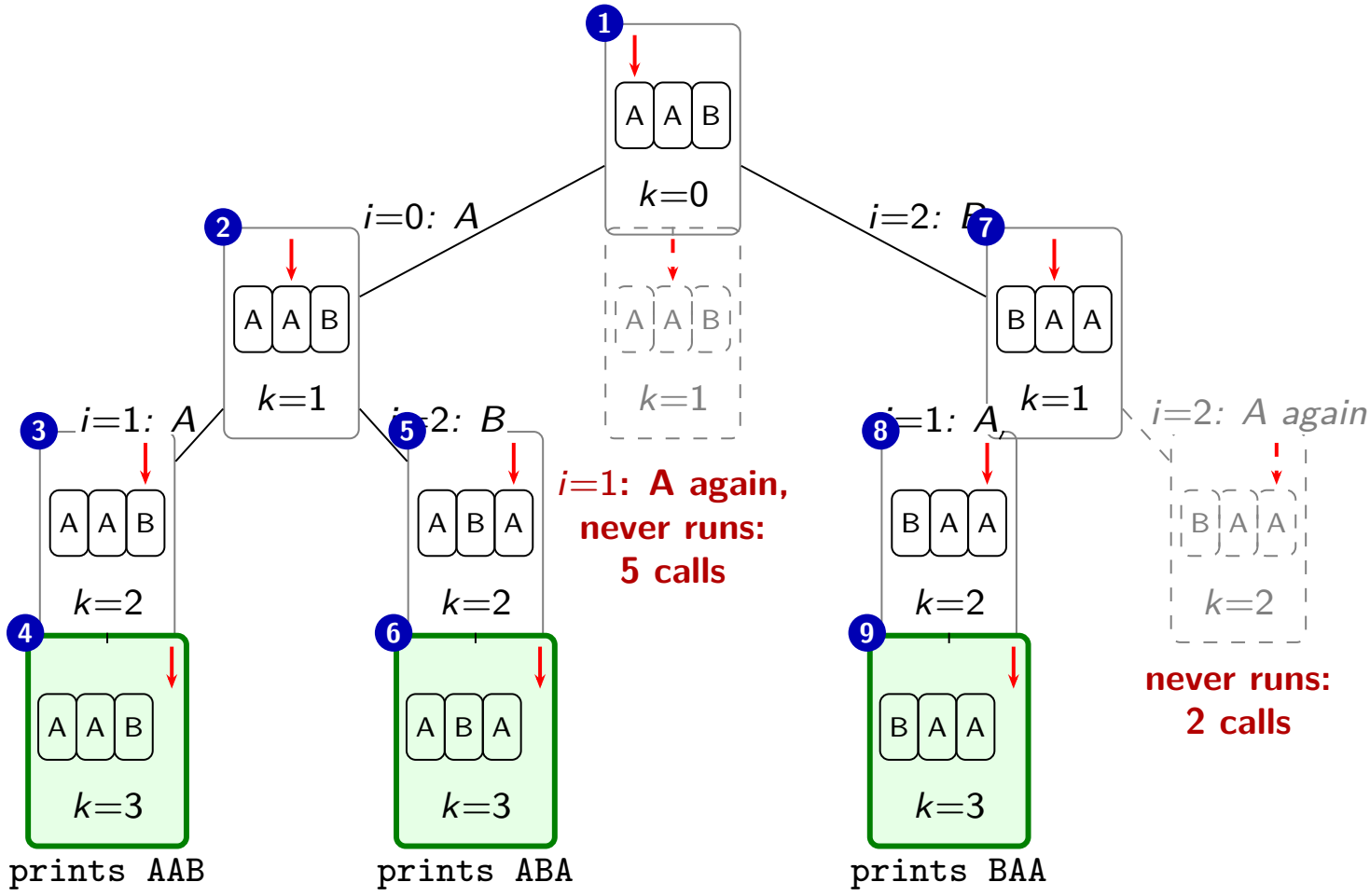
Each node: the string with an arrow at seat `k`. Edge: which letter goes into seat `k` (`i` is the position it comes from).



9 calls, numbered in the order they run. Each green leaf (`k = 3`) prints one string.

The Call Tree of `printUnique("AAB", 0)`

Each node: the string with an arrow at seat `k`. Edge: which letter goes into seat `k` (`i` is the position it comes from).



The set `tried` stops the repeated letters. A skipped branch also skips everything below it: $9 + 5 + 2 = \mathbf{16}$ calls for `printAll`, only **9** for `printUnique`.

Common Mistakes

- **Pruning on an unproven reason**: a negative `rem` is a dead end only for non-negative numbers.
- **Ignoring the result of the recursive call**: `solve(rest);` instead of `if (solve(rest)) return true;` finds the answer and then throws it away.
- **Returning before the undo**: any `return` between choose and undo leaves the state modified. Returning before undoing shared state is safe only if the caller does not require that state to be restored.
- **Stopping early when asked to count**: "how many solutions?" needs the *whole* tree.
- **Comparing `doubles` with `==`** in the goal test: use a tolerance. Rounded keys in a `seen` set are approximate, so treat them with care.
- **Sharing one `seen` set between different problems**: clear it, or make a new one, for each start.

Practice

1. A take/skip tree over 9 numbers has how many nodes? How many are leaves?
2. In the bound test, why does `rem > suffix[k]` guarantee failure?
3. How many different arrangements does `"AABB"` have?
4. Give a reason why the prune `rem < 0` would be **wrong** for this input: `v = {6, -2, 1}`, target 5.
5. In Make 24, why do we try both $a - b$ and $b - a$, but only one of $a + b$ and $b + a$?

Practice

1. A take/skip tree over 9 numbers has how many nodes? How many are leaves?
2. In the bound test, why does `rem > suffix[k]` guarantee failure?
3. How many different arrangements does `"AABB"` have?
4. Give a reason why the prune `rem < 0` would be **wrong** for this input: `v = {6, -2, 1}`, target 5.
5. In Make 24, why do we try both $a - b$ and $b - a$, but only one of $a + b$ and $b + a$?

1. $2^{10} - 1 = 1023$ nodes; $2^9 = 512$ leaves.
2. Even taking **every** remaining number gives a sum of only `suffix[k]`, which is less than `rem`. (This needs non-negative numbers.)
3. $4!/(2!\,2!) = 6$.
4. Taking 6 gives `rem` $= -1$, which is cut. But $6 + (-2) + 1 = 5$, so the prune hides a real solution.
5. $a + b = b + a$, but subtraction and division are generally not commutative, so both operand orders must be considered ($a - b$ and $b - a$; $a \div b$ and $b \div a$).

Summary and Preview

- **Early termination:** stop at the first answer when the question is "does one exist?"
- **Pruning:** skip a branch only when you can **prove** it holds no goal. A **bound** (`rem > suffix[k]`) catches impossible targets at once.
- **Symmetry:** do not try $a + b$ and $b + a$ as separate moves.
- **Duplicate states:** remember what you have seen; use a canonical (sorted) state key; `double` keys need special care.
- **Project 3** combines all of these: the handout lists what your solver must do.

Coming up

Nov 3: Exercise 7 due; Exercise 8 (Shunting-Yard Trace) posted.
Nov 3: Expression parsing: infix, postfix, and the shunting-yard algorithm. The stack and queue from earlier weeks return, to turn $3 + 4 \times 2$ into something a computer can evaluate.